

[Ostfalia Hochschule für angewandte Wissenschaften]

[Digitale Technologie]

**Ostfalia**  
Hochschule für angewandte  
Wissenschaften



Masterarbeit zum Thema:

**Konzeption und Umsetzung einer webbasierten  
Benutzeroberfläche für eine hybride  
Verkehrssimulationsumgebung**

Vorgelegt von:

Name: Nam Nguyen

Kontaktdaten: na.nguyen@ostfalia.de

Matrikelnummer: 70487857

Dozent/Dozentin: Prof. Dr.-Ing. Tamás Kurczveil

Zweitprüfer: Prof. Dr. sc. ETH Gerko Santel

Abgabedatum: 31. Mai 2026

# Inhaltsverzeichnis

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| <b>Abbildungsverzeichnis .....</b>                                            | <b>I</b>   |
| <b>Tabellenverzeichnis .....</b>                                              | <b>III</b> |
| <b>Abkürzungsverzeichnis .....</b>                                            | <b>IV</b>  |
| <b>1. Einleitung .....</b>                                                    | <b>1</b>   |
| 1.1 Motivation .....                                                          | 2          |
| 1.2 Problemstellung & Forschungsfrage .....                                   | 3          |
| 1.3 Zielsetzung der Arbeit .....                                              | 5          |
| 1.4 Aufbau der Arbeit .....                                                   | 6          |
| <b>2. Grundlagen .....</b>                                                    | <b>7</b>   |
| 2.1 Verkehrssimulation – Ziele, Methoden und Einsatzgebiete .....             | 7          |
| 2.2 Überblick über existierende Simulationstools (SUMO, VISSIM, Aimsun) ..... | 8          |
| 2.3 Hybride Simulationsarchitekturen .....                                    | 9          |
| 2.4 Webtechnologien für Visualisierung und Interaktion .....                  | 11         |
| 2.4.1 React .....                                                             | 12         |
| 2.4.2 MapTiler / Kartenvisualisierung .....                                   | 12         |
| 2.4.3 REST-APIs & WebSockets .....                                            | 13         |
| <b>3. Methodisches Vorgehen .....</b>                                         | <b>14</b>  |
| <b>4. Analyse der Anforderungen .....</b>                                     | <b>16</b>  |
| 4.1 Funktionale Anforderungen .....                                           | 16         |
| 4.2 Nichtfunktionale Anforderungen .....                                      | 18         |
| 4.3 Nutzerrollen & Anwendungskontext .....                                    | 19         |
| <b>5. Konzeption und Systemarchitektur .....</b>                              | <b>20</b>  |
| 5.1 Ziele der GUI .....                                                       | 20         |
| 5.2 Layout der Benutzeroberfläche .....                                       | 21         |
| 5.3 Zentrale GUI-Komponenten .....                                            | 22         |

|           |                                                             |           |
|-----------|-------------------------------------------------------------|-----------|
| 5.3.1     | Werkzeugleiste .....                                        | 22        |
| 5.3.2     | Kartenansicht.....                                          | 22        |
| 5.3.3     | Fahrzeugmanagement.....                                     | 23        |
| 5.3.4     | Dialogsystem .....                                          | 24        |
| 5.4       | Systemarchitektur .....                                     | 25        |
| 5.4.1     | Frontend.....                                               | 25        |
| 5.4.2     | Backend (FastAPI) .....                                     | 27        |
| 5.4.3     | Simulationsengines.....                                     | 27        |
| 5.4.4     | Datenbank.....                                              | 27        |
| 5.5       | Kommunikation zwischen Frontend und Backend.....            | 28        |
| 5.6       | Struktur der Simulationsdaten .....                         | 29        |
| 5.7       | Docker-basierte Infrastruktur .....                         | 30        |
| <b>6.</b> | <b>Implementierung .....</b>                                | <b>32</b> |
| 6.1       | Implementierung des Frontends .....                         | 32        |
| 6.1.1     | Kartenkomponente (MapTiler-Integration).....                | 34        |
| 6.1.2     | Werkzeugleiste .....                                        | 35        |
| 6.1.3     | Fahrzeugverwaltung .....                                    | 36        |
| 6.1.4     | Fahrzeugkonfiguration.....                                  | 37        |
| 6.2       | Implementierung des Backends.....                           | 38        |
| 6.2.1     | API-Struktur .....                                          | 39        |
| 6.2.2     | Simulationssteuerung .....                                  | 40        |
| 6.2.3     | Abstraktion der Simulationsengine .....                     | 41        |
| 6.3       | Implementierung der internen Simulation.....                | 42        |
| 6.4       | Integration einer externen Simulationsengine mit SUMO ..... | 43        |
| 6.4.1     | Kommunikation über TraCI .....                              | 45        |
| 6.4.2     | Implementierung der SUMO-Anbindung.....                     | 46        |

|           |                                                                    |            |
|-----------|--------------------------------------------------------------------|------------|
| 6.5       | Vergleich der Simulationsvarianten und Erweiterungspotenziale..... | 46         |
| <b>7.</b> | <b>Evaluation .....</b>                                            | <b>48</b>  |
| 7.1       | Funktionale Tests.....                                             | 48         |
| 7.2       | Explorativer Usability-Test .....                                  | 50         |
| 7.3       | Vergleich GUI vs. SUMO-GUI.....                                    | 53         |
| 7.4       | Diskussion der Ergebnisse.....                                     | 55         |
| <b>8.</b> | <b>Fazit .....</b>                                                 | <b>56</b>  |
|           | <b>Literaturverzeichnis .....</b>                                  | <b>IV</b>  |
|           | <b>Eidesstattliche Erklärung .....</b>                             | <b>VII</b> |

## Abbildungsverzeichnis

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Abbildung 1: Hybride Architektur der Verkehrssimulation.....                      | 10 |
| Abbildung 2: Webtechnologien für Visualisierung und Interaktion .....             | 11 |
| Abbildung 3: REST APIs & WebSockets .....                                         | 13 |
| Abbildung 4: Iterativer Entwicklungsprozess .....                                 | 15 |
| Abbildung 5: Nutzerrollen & Anwendungskontext .....                               | 19 |
| Abbildung 6: Benutzeroberfläche der GUI .....                                     | 21 |
| Abbildung 7: Werkzeugleiste (Steuerung der Simulation).....                       | 22 |
| Abbildung 8: Kartenansicht (MapTiler) .....                                       | 22 |
| Abbildung 9: Fahrzeugmanagement.....                                              | 23 |
| Abbildung 10: Dialogsystem (Fahrzeugtypen) .....                                  | 24 |
| Abbildung 11: Gesamtarchitektur der Verkehrssimulationsplattform .....            | 26 |
| Abbildung 12: Datenmodell .....                                                   | 29 |
| Abbildung 13: Container Architektur.....                                          | 30 |
| Abbildung 14: Docker (Container).....                                             | 31 |
| Abbildung 15: Die wichtigsten Module für die Verkehrssimulation .....             | 33 |
| Abbildung 16: MapContainer.tsx (wichtigstes Modul der Kartenintegration).....     | 34 |
| Abbildung 17: AppToolbar.tsx (zentrales Modul für Werkzeugleiste).....            | 35 |
| Abbildung 18: VehiclePopup.tsx (wichtigste Modul für Fahrzeugkonfiguration).....  | 36 |
| Abbildung 19: VehicleLayer.ts (wichtigste Modul für die Fahrzeugverwaltung) ..... | 37 |
| Abbildung 20: main.py (Hauptdatei der FastAPI-Backendanwendung).....              | 38 |
| Abbildung 21: Struktur der REST-API .....                                         | 39 |
| Abbildung 22: Module für die Simulationsengines.....                              | 40 |
| Abbildung 23: Ablaufdiagramm der internen Simulation .....                        | 42 |
| Abbildung 24: SUMO-Integration in einer Docker Umgebung.....                      | 44 |

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Abbildung 25: Kommunikationsablauf zwischen Backend und SUMO über TraCI ..... | 45 |
| Abbildung 26: Vergleich zwischen interner Simulation und SUMO .....           | 47 |

## Tabellenverzeichnis

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Tabelle 1: Anwendung des DSR-Rahmenwerks auf die vorliegende Arbeit ..... | 14 |
| Tabelle 2: Funktionale Anforderungen (Tabellenform): .....                | 16 |
| Tabelle 3: Nichtfunktionale Anforderungen (Tabellenform): .....           | 18 |
| Tabelle 4: Funktionale Testergebnisse .....                               | 49 |
| Tabelle 5: UMUX-Lite Ergebnisse .....                                     | 51 |
| Tabelle 6: Vergleich GUI vs. SUMO-GUI.....                                | 53 |

## Abkürzungsverzeichnis

|           |                                                    |
|-----------|----------------------------------------------------|
| API       | Application Programming Interface                  |
| FastAPI   | Python-Webframework für APIs                       |
| GUI       | Graphical User Interface                           |
| HTTP      | Hypertext Transfer Protocol                        |
| HTTPS     | Hypertext Transfer Protocol Secure                 |
| JSON      | JavaScript Object Notation                         |
| KI        | Künstliche Intelligenz                             |
| KPI       | Key Performance Indicator                          |
| LSA       | Lichtsignalanlage                                  |
| OSM       | OpenStreetMap                                      |
| REST      | Representational State Transfer                    |
| SUMO      | Simulation of Urban MObility                       |
| TCP       | Transmission Control Protocol                      |
| TraCI     | Traffic Control Interface                          |
| UI        | User Interface                                     |
| URL       | Uniform Resource Locator                           |
| WebSocket | Protokoll für bidirektionale Echtzeitkommunikation |
| XML       | Extensible Markup Language                         |



## **Abstract**

Verkehrssimulationen sind ein zentrales Werkzeug der Verkehrsforschung, um komplexe Verkehrsprozesse zu analysieren und verkehrliche Maßnahmen unter kontrollierten Bedingungen zu untersuchen. Etablierte Simulationswerkzeuge wie SUMO (Simulation of Urban Mobility) bieten leistungsfähige Modelle zur Abbildung von Verkehrsdynamiken, sind jedoch häufig technisch komplex und primär auf erfahrene Anwender ausgerichtet. Dies erschwert insbesondere den Einsatz in Lehrveranstaltungen, prototypischen Forschungsarbeiten und Laborumgebungen, in denen eine schnelle und interaktive Nutzung erforderlich ist.

Ziel dieser Arbeit ist die Konzeption und prototypische Umsetzung einer hybriden Verkehrssimulationsumgebung mit einer webbasierten, benutzerfreundlichen Benutzeroberfläche. Die Lösung ermöglicht die interaktive Konfiguration, Steuerung und Visualisierung von Verkehrsszenarien und unterstützt sowohl eine einfach implementierte interne Simulation als auch die Anbindung einer externen Simulationsengine wie SUMO. Der Fokus liegt dabei auf der Reduktion technischer Komplexität und der Verbesserung der Nutzerinteraktion. Darüber hinaus ist die Benutzeroberfläche auf den Einsatz in Indoor-Laborumgebungen ausgerichtet und bildet perspektivisch die Grundlage für die spätere Kopplung von Simulationen mit realer Hardware. Die Ergebnisse dieser Arbeit zeigen, dass durch eine leichtgewichtige, webbasierte Benutzeroberfläche der Zugang zu Verkehrssimulationen vereinfacht und neue Anwendungsmöglichkeiten in Lehre, Forschung und Laborbetrieb eröffnet werden können.

# 1. Einleitung

Verkehr setzt sich aus vielen einzelnen Fahrzeugen zusammen, die gleichzeitig unterwegs sind und sich gegenseitig beeinflussen. Bereits geringfügige Änderungen, wie ein langsames Fahrzeug oder eine zusätzliche Phase einer Lichtsignalanlage, können erhebliche Auswirkungen haben – etwa Staus oder längere Wartezeiten. Deshalb lässt sich Verkehr nicht allein durch Beobachtung oder Erfahrung zuverlässig vorhersagen. In der Verkehrsforschung werden Verkehrssimulationen eingesetzt, um den Verkehr im Computer modellhaft nachzubilden und gezielt zu untersuchen, wie sich bestimmte Änderungen auf den Verkehr auswirken können. Ein wichtiger Grund für den Einsatz von Verkehrssimulationen ist die Sicherheit. Viele Fragestellungen im Verkehr können im realen Straßenverkehr nicht getestet werden, weil sie zu gefährlich wären. Beispiele hierfür wären reale Tests mit Notbremsungen oder plötzlichen Manövern von Fahrzeugen. Außerdem sind reale Verkehrsversuche sehr aufwendig und teuer. Solche Versuche würden Straßensperrungen, Umleitungen und den Einsatz von zusätzlichem Personal erfordern. Verkehrssimulationen bieten hier eine flexible und kostengünstige Alternative, da verschiedene Varianten schnell und mit wenig Aufwand ausprobiert werden können. Dadurch ist es möglich unterschiedliche Signalprogramme von Lichtsignalanlagen oder verschiedene Routenführungen miteinander zu vergleichen, ohne in den realen Verkehr einzugreifen. Ein Vorteil von Verkehrssimulationen ist, dass sie unter gleichen Bedingungen wiederholt werden können. Im echten Verkehr ändern sich ständig äußere Einflüsse wie Wetter, Verkehrsaufkommen oder individuelles Fahrverhalten. In einer Simulation kann dieselbe Situation jedoch mehrfach exakt gleich nachgestellt werden, was die systematische Analyse einzelner Änderungen erlaubt.<sup>3</sup> Nicht zuletzt sind Verkehrssimulationen auch in Forschung und Lehre von großer Bedeutung. Sie machen Verkehrsabläufe sichtbar und verständlich und helfen dabei, komplexe Zusammenhänge anschaulich zu erklären, etwa wie Stau entsteht oder warum sich bestimmte Verkehrsmaßnahmen unterschiedlich auswirken.<sup>1/2</sup>

---

<sup>1</sup> Baza-Solares, N.-M. R.-B. (kein Datum). Traffic Simulation with Open-Source and Commercial Traffic Microsimulators: A Case Study.

<sup>2</sup> Bundesanstalt für Straßen- und Verkehrswesen. (kein Datum). Mikroskopische Verkehrsflusssimulation auf einbahnigen Landstraßen.

## 1.1 Motivation

Verkehrssimulationen sind ein etabliertes Hilfsmittel der Verkehrsplanung und Verkehrsforschung. Systeme wie SUMO, PTV VISSIM oder Aimsun ermöglichen die Modellierung und Analyse komplexer Verkehrsnetze. Sie werden unter anderem genutzt, um infrastrukturelle Maßnahmen zu bewerten, Verkehrssteuerungen zu optimieren und dynamische Verkehrsprozesse unter kontrollierten Bedingungen zu untersuchen. Neben großen Planungs- und Forschungsprojekten gibt es jedoch auch viele Anwendungsszenarien, in denen nicht die detaillierte Modellierung komplexer Netze im Mittelpunkt steht. Stattdessen besteht hier der Bedarf, einzelne Verkehrssituationen flexibel, interaktiv und schnell untersuchen zu können. Dies gilt insbesondere für Lehrveranstaltungen, Laborumgebungen und prototypische Forschungsarbeiten, in denen Verkehrsszenarien einfach konfiguriert, direkt visualisiert und experimentell verändert werden sollen.<sup>3/4</sup>

Vor diesem Hintergrund wird in dieser Arbeit eine anwendungsorientierte Verkehrssimulationsumgebung entwickelt. Ziel ist die Entwicklung einer benutzerfreundlichen, webbasierten Benutzeroberfläche, die eine intuitive Interaktion mit Verkehrssimulationen ermöglicht und deren Nutzung erleichtert. Die Plattform soll sowohl eine einfache interne Simulation unterstützen, die als vereinfachte Simulationslogik im Backend umgesetzt ist, als auch die Anbindung einer externen Simulationsengine wie SUMO.<sup>5</sup> Die externe Simulationsengine übernimmt dabei die Berechnung der Verkehrsdynamik und stellt die Fahrzeugzustände für die Visualisierung bereit. Ein weiterer Grund für diese Arbeit ist der geplante Einsatz in Indoor-Laborumgebungen. Die Plattform soll perspektivisch nicht nur virtuelle Simulationen steuern und visualisieren, sondern auch die spätere Einbindung realer Hardwarekomponenten wie Modellfahrzeuge oder mobiler Roboter unterstützen. Dadurch können digitale Simulationen mit physischen Experimenten verbunden und innerhalb einer gemeinsamen Umgebung gesteuert und überwacht werden. Die Motivation dieser Arbeit liegt somit in der Entwicklung einer flexiblen und interaktiven Benutzeroberfläche, die Verkehrssimulationen für Lehre, Labor und prototypische Forschung leichter zugänglich macht und zugleich hybride Simulationsansätze unterstützt.<sup>6</sup>

---

<sup>3</sup> aimsun. (kein Datum). Aimsun delivers software and services to support transport planning and operations.

<sup>4</sup> PTV Group. (kein Datum). Verkehrssimulation Software.

<sup>5</sup> Simulation of Urban MObility. (kein Datum). SUMO.

<sup>6</sup> Bundesanstalt für Straßen- und Verkehrswesen. (kein Datum). Fahrsimulator.

## 1.2 Problemstellung & Forschungsfrage

Trotz der hohen Leistungsfähigkeit etablierter Verkehrssimulationsprogramme zeigt sich in bestimmten Anwendungsszenarien ein deutlicher Unterschied zwischen Funktionsumfang und praktischer Nutzbarkeit. Systeme wie SUMO, VISSIM oder Aimsun wurden für die detaillierte Analyse komplexer Verkehrsnetze entwickelt. In der Regel erfordern sie jedoch technisches Vorwissen sowie eine vorherige Konfiguration über spezialisierte Benutzeroberflächen oder Konfigurationsdateien.<sup>7</sup>

Die Erstellung und Anpassung von Szenarien ist häufig mit einem erhöhten Aufwand verbunden. Änderungen an Parametern, Fahrzeugverhalten oder Simulationsabläufen können oftmals nur vor dem Start einer Simulation vorgenommen werden. Eine flexible und interaktive Anpassung während der laufenden Simulation ist dagegen nur eingeschränkt möglich. Dies erschwert experimentelles Ausprobieren und spontane Änderungen, wie sie insbesondere in Lehrveranstaltungen oder prototypischen Forschungsarbeiten erforderlich sind.<sup>8</sup>

Hinzu kommt, dass viele bestehende Simulationsprogramme eine lokale Installation sowie spezifische Systemkonfigurationen voraussetzen. Dadurch wird der plattformunabhängige und kollaborative Einsatz eingeschränkt. Eine leicht zugängliche, webbasierte Oberfläche zur Steuerung und Visualisierung von Simulationen, die unterschiedliche Simulationsquellen integrieren kann, ist bislang nur begrenzt verfügbar.<sup>9</sup>

Ein weiterer Aspekt betrifft hybride Anwendungsszenarien in Indoor-Laborumgebungen. Für experimentelle Untersuchungen mit Modellfahrzeugen oder mobilen Robotern ist eine direkte Verbindung zwischen Simulation und realer Hardware erforderlich. Bestehende Verkehrssimulationssysteme bieten hierfür jedoch keine einheitliche, interaktive und benutzerfreundliche Schnittstelle, die virtuelle und reale Komponenten gemeinsam steuern und visualisieren kann.<sup>10</sup>

---

<sup>7</sup> Simulation of Urban MObility. (kein Datum). SUMO.

<sup>8</sup> Daniel Krajzewicz, J. E. (kein Datum). Recent Development and Applications of SUMO – Simulation of Urban MObility.

<sup>9</sup> Simulation of Urban MObility. (kein Datum). SUMO.

<sup>10</sup> Tellmann, D. (kein Datum). Hardware-in-the-Loop-gestützte Entwicklungsplattform für Fahrerassistenzsysteme.

Aus diesen Beobachtungen ergibt sich die zentrale Forschungsfrage dieser Arbeit:

- **Wie kann eine einfache Benutzeroberfläche einer Verkehrssimulationsssoftware (SUMO) entwickelt und gestaltet werden, die nicht-spezialisierte Nutzer in Lehr- und Laborumgebungen unterstützt?**

Diese Fragestellung greift die zuvor beschriebene Lücke zwischen leistungsfähigen, aber komplexen Verkehrssimulationssystemen und dem Bedarf an zugänglichen, flexiblen und interaktiven Anwendungen auf. Im Mittelpunkt steht dabei nicht der Ersatz von SUMO, sondern die Entwicklung einer webbasierten Benutzeroberfläche, die SUMO für nicht-spezialisierte Nutzer zugänglich macht und gleichzeitig den Einsatz in Lehr- und Laborumgebungen ermöglicht. Ziel ist die Entwicklung einer Architektur, die Interaktivität, Modularität und Hybridfähigkeit miteinander verbindet.

## 1.3 Zielsetzung der Arbeit

Ziel dieser Arbeit ist der Entwurf und die prototypische Implementierung einer webbasierten Verkehrssimulationsplattform mit hybrider Architektur. Im Mittelpunkt steht die Entwicklung einer modularen Benutzeroberfläche, die als zentrale Oberfläche zur Steuerung und Visualisierung zwischen Nutzer, Simulationslogik und optionalen externen Komponenten dient. Das System soll eine vereinfachte interne Simulation bereitstellen und gleichzeitig die Integration einer externen Simulationsengine ermöglichen. Dafür wird eine einheitliche Schnittstelle entwickelt, über die unterschiedliche Simulationsengines auf die gleiche Weise angesteuert werden können.

Ein weiterer Bestandteil der Zielsetzung ist die Entwicklung einer klar strukturierten Systemarchitektur mit getrennten Aufgabenbereichen für die Benutzeroberfläche (Frontend), die serverseitige Verarbeitung (Backend) und die eigentliche Simulationsberechnung. Dadurch sollen Modularität, Erweiterbarkeit und Wartbarkeit des Systems gewährleistet werden. Darüber hinaus wird die Plattform so konzipiert, dass sie perspektivisch die Verbindung virtueller Simulationen mit realen Hardwarekomponenten unterstützt. Die Oberfläche dient dabei als zentrale Schnittstelle, über die sowohl digitale als auch physische Systemkomponenten gesteuert werden können. Eine konkrete Anbindung realer Hardware ist jedoch nicht Gegenstand der prototypischen Umsetzung dieser Arbeit.

Die Arbeit verfolgt somit das Ziel, eine technische Architektur und einen funktionsfähigen Prototyp zu entwickeln, der Interaktivität, Engine-Unabhängigkeit und Hybridfähigkeit in einer webbasierten Umgebung vereint. Im Mittelpunkt steht dabei nicht die vollständige Abbildung aller Funktionalitäten etablierter Verkehrssimulationswerkzeuge, sondern die prototypische Entwicklung und Untersuchung einer modularen webbasierten Benutzeroberfläche. Komplexe Verkehrsmodelle, umfangreiche Netzbearbeitung sowie detaillierte Analysefunktionen externer Simulationssysteme sind nicht Gegenstand der prototypischen Umsetzung. Der Fokus liegt vielmehr auf der Konzeption einer flexiblen und erweiterbaren Architektur für interaktive Verkehrssimulationen.

## 1.4 Aufbau der Arbeit

Die vorliegende Arbeit ist in mehrere aufeinander aufbauende Kapitel gegliedert. Ziel dieser Struktur ist es, die Entwicklung der webbasierten Verkehrssimulationsplattform Schritt für Schritt darzustellen – von den theoretischen Grundlagen über Konzeption und Implementierung bis hin zur Bewertung der Ergebnisse.

Zu Beginn werden in Kapitel 2 die theoretischen und technischen Grundlagen der Arbeit erläutert. Dazu gehören zentrale Konzepte der Verkehrssimulation, ein Überblick über bestehende Simulationswerkzeuge sowie eine Einführung in hybride Simulationsarchitekturen und die eingesetzten Webtechnologien.

Kapitel 3 beschreibt anschließend das methodische Vorgehen der Arbeit. Es stellt den prototypischen Entwicklungsprozess sowie das Konzept zur späteren Bewertung der entwickelten Plattform vor. Darauf aufbauend widmet sich Kapitel 4 der Analyse der Anforderungen. Hier werden sowohl funktionale als auch nichtfunktionale Anforderungen der Plattform sowie relevante Nutzerrollen und Anwendungsszenarien definiert.

Auf Grundlage dieser Anforderungen wird in Kapitel 5 die Konzeption der webbasierten Benutzeroberfläche sowie die technische Systemarchitektur vorgestellt. Dabei werden das Layout der Oberfläche, zentrale GUI-Komponenten sowie das Zusammenspiel von Frontend, Backend, Simulationsengines, Datenmodell und Docker-Infrastruktur beschrieben. Die konkrete technische Umsetzung der Plattform wird in Kapitel 6 beschrieben. Dort werden die Implementierung des Frontends, des Backends, der internen Simulation sowie die Integration der externen Simulationsengine SUMO erläutert. Dabei werden sowohl die technischen Grundlagen von TraCI als auch die Einbindung von SUMO in die hybride Architektur dargestellt.

Die Bewertung der entwickelten Plattform erfolgt in Kapitel 7. Hier werden die Ergebnisse der funktionalen Tests und des explorativen Usability-Tests vorgestellt sowie ein qualitativer Vergleich mit der bestehenden SUMO-GUI durchgeführt.

Abschließend fasst Kapitel 8 die wichtigsten Ergebnisse der Arbeit zusammen und gibt einen Ausblick auf mögliche zukünftige Erweiterungen.

## 2. Grundlagen

Das folgende Kapitel stellt die theoretischen und technischen Grundlagen der Arbeit vor. Es bildet die theoretische Grundlage für die Entwicklung der hybriden Verkehrssimulationsumgebung. Zunächst werden grundlegende Konzepte der Verkehrssimulation erläutert und überblicksartig dargestellt. Anschließend wird das Grundprinzip hybrider Simulationsarchitekturen beschrieben. Darüber hinaus werden die verwendeten Webtechnologien vorgestellt, die die technische Basis der entwickelten Plattform bilden. Dieses Kapitel bildet damit die Grundlage für die nachfolgenden Kapitel, in denen die Konzeption, die technische Umsetzung und die Bewertung der Plattform dargestellt werden.

### 2.1 Verkehrssimulation – Ziele, Methoden und Einsatzgebiete

Eine Verkehrssimulation ist eine computergestützte Nachbildung von Verkehrsabläufen. Dabei wird das Verhalten von Fahrzeugen und Elementen der Verkehrsinfrastruktur in einem digitalen Modell dargestellt. Ziel ist es, Verkehrsprozesse unter kontrollierten Bedingungen zu analysieren und zu bewerten. Viele Fahrzeuge bewegen sich gleichzeitig durch ein Straßennetz und beeinflussen sich gegenseitig. Zusätzlich wirken Infrastrukturelemente wie Lichtsignalanlagen, Kreuzungen oder Fahrspuren auf das Verkehrsgeschehen ein. Bereits kleine Änderungen – etwa eine veränderte Phase einer Lichtsignalanlage – können deutliche Auswirkungen auf den Verkehrsfluss haben.<sup>11</sup>

Da reale Experimente im Straßenverkehr häufig teuer, aufwendig oder sicherheitskritisch sind, werden Verkehrssimulationen eingesetzt. Sie ermöglichen es, verschiedene Szenarien virtuell zu testen und miteinander zu vergleichen. Ein großer Vorteil besteht darin, dass Simulationen unter identischen Bedingungen wiederholt werden können.<sup>12</sup>

---

<sup>11</sup> Fraunhofer ITWM. (kein Datum). Verkehrssimulation.

<sup>12</sup> Hochschule Liechtenstein. (kein Datum). Simulation informationsorientierter Verkehrssysteme.



## 2.2 Überblick über existierende Simulationstools (SUMO, VISSIM, Aimsun)

Im Bereich der Verkehrssimulation gibt es verschiedene etablierte Softwarelösungen, die weltweit in Forschung, Planung und Industrie eingesetzt werden. Zu den bekanntesten Simulationsprogrammen zählen SUMO (Simulation of Urban MObility), PTV Vissim und Aimsun. Diese Systeme ermöglichen die detaillierte Modellierung und Analyse von Verkehrsnetzen und basieren überwiegend auf mikroskopischen Verkehrssimulationen. Bei mikroskopischen Verkehrssimulationen werden einzelne Fahrzeuge als eigenständige Objekte modelliert. Jedes Fahrzeug besitzt individuelle Bewegungs- und Interaktionsregeln. Dadurch können Fahrzeugdynamik, Spurwechselvorgänge, Routenentscheidungen sowie Interaktionen zwischen Verkehrsteilnehmenden realitätsnah abgebildet werden.<sup>13</sup>

SUMO ist eine frei verfügbare und in der Forschung weit verbreitete Verkehrssimulationssoftware. Sie unterstützt die Simulation komplexer Verkehrsnetze und bietet mit der Schnittstelle TraCI (Traffic Control Interface) die Möglichkeit, eine laufende Simulation von außen zu steuern.<sup>14</sup>

PTV Vissim und Aimsun sind kommerzielle Simulationsprogramme mit umfangreichen Modellierungs- und Analysefunktionen. Sie werden vor allem in der Verkehrsplanung sowie bei Entscheidungen über Verkehrsinfrastruktur eingesetzt. Beide Systeme bieten leistungsfähige grafische Benutzeroberflächen sowie detaillierte Auswertungsmöglichkeiten.<sup>15/16</sup>

In der vorliegenden Arbeit wird SUMO als externe Simulationsengine in die entwickelte Plattform integriert. Die Auswahl erfolgt aufgrund der offenen Schnittstellen, der weiten Verbreitung in der Forschung sowie der Möglichkeit, Simulationen programmgesteuert über TraCI zu steuern.<sup>17/18</sup>

---

<sup>13</sup> Daniel Krajzewicz, J. E. (kein Datum). Recent Development and Applications of SUMO – Simulation of Urban MObility.

<sup>14</sup> Simulation of Urban MObility. (kein Datum). SUMO.

<sup>15</sup> aimsun. (kein Datum). Aimsun delivers software and services to support transport planning and operations.

<sup>16</sup> PTV Group. (kein Datum). Verkehrssimulation Software.

<sup>17</sup> Daniel Krajzewicz, J. E. (kein Datum). Recent Development and Applications of SUMO – Simulation of Urban MObility.

<sup>18</sup> Simulation of Urban MObility. (kein Datum). SUMO.

## 2.3 Hybride Simulationsarchitekturen

Eine hybride Simulationsarchitektur beschreibt ein System, in dem eine digitale Simulation mit realer Hardware kombiniert werden kann. Die Simulation läuft dabei nicht ausschließlich virtuell am Computer, sondern wird perspektivisch mit realen Komponenten verbunden. Ziel ist es, virtuelle und reale Elemente in einem gemeinsamen System zusammenzuführen.<sup>19</sup>

In dieser Arbeit bedeutet „hybrid“ zunächst, dass die entwickelte webbasierte Benutzeroberfläche unterschiedliche Simulationsquellen innerhalb einer gemeinsamen Plattform integriert. Neben einer rein virtuellen Verkehrssimulation ist die Architektur so ausgelegt, dass zukünftig auch reale Modellfahrzeuge oder mobile Roboter eingebunden werden können. Eine direkte Kopplung zwischen Simulation und realer Hardware wurde im Rahmen dieser Arbeit jedoch noch nicht umgesetzt, sondern lediglich konzeptionell berücksichtigt.<sup>20</sup>

Der Vorteil dieses Ansatzes liegt darin, dass Experimente unter kontrollierten Bedingungen vorbereitet werden können. Verkehrsszenarien lassen sich zunächst virtuell planen und testen. Perspektivisch können einzelne Fahrzeuge oder Komponenten in Laborumgebungen real eingesetzt und mit der Simulation gekoppelt werden. Dadurch entsteht eine flexible experimentelle Umgebung, die sowohl virtuelle Analysen als auch praktische Experimente ermöglichen kann.<sup>21</sup>

Für diese Arbeit bedeutet dies, dass die Benutzeroberfläche als zentrale Oberfläche zur Steuerung und Visualisierung dient. Sie verwaltet die interne Simulation sowie externe Simulationsengines und ist architektonisch für eine spätere Anbindung realer Hardware vorbereitet. Eine klare Trennung zwischen Benutzeroberfläche, Simulationslogik und potenziellen Schnittstellen zur Hardware ist dabei entscheidend, um das System erweiterbar und stabil zu gestalten.

---

<sup>19</sup> Reinold, D.-I. P. (kein Datum). Verkehrssimulation im Hardware-in-the-Loop-Steuergerätetest.

<sup>20</sup> Tellmann, D. (kein Datum). Hardware-in-the-Loop-gestützte Entwicklungsplattform für Fahrerassistenzsysteme.

<sup>21</sup> Kim, Y. (kein Datum). Hardware-In-the-Loop for Connected Automated Vehicles Testing inReal Traffic.

Die entwickelte Architektur schafft somit die Grundlage für einen zukünftigen Einsatz der Simulationsplattform in Indoor-Laboren, in denen virtuelle Verkehrsszenarien mit realen Modellfahrzeugen kombiniert werden können.

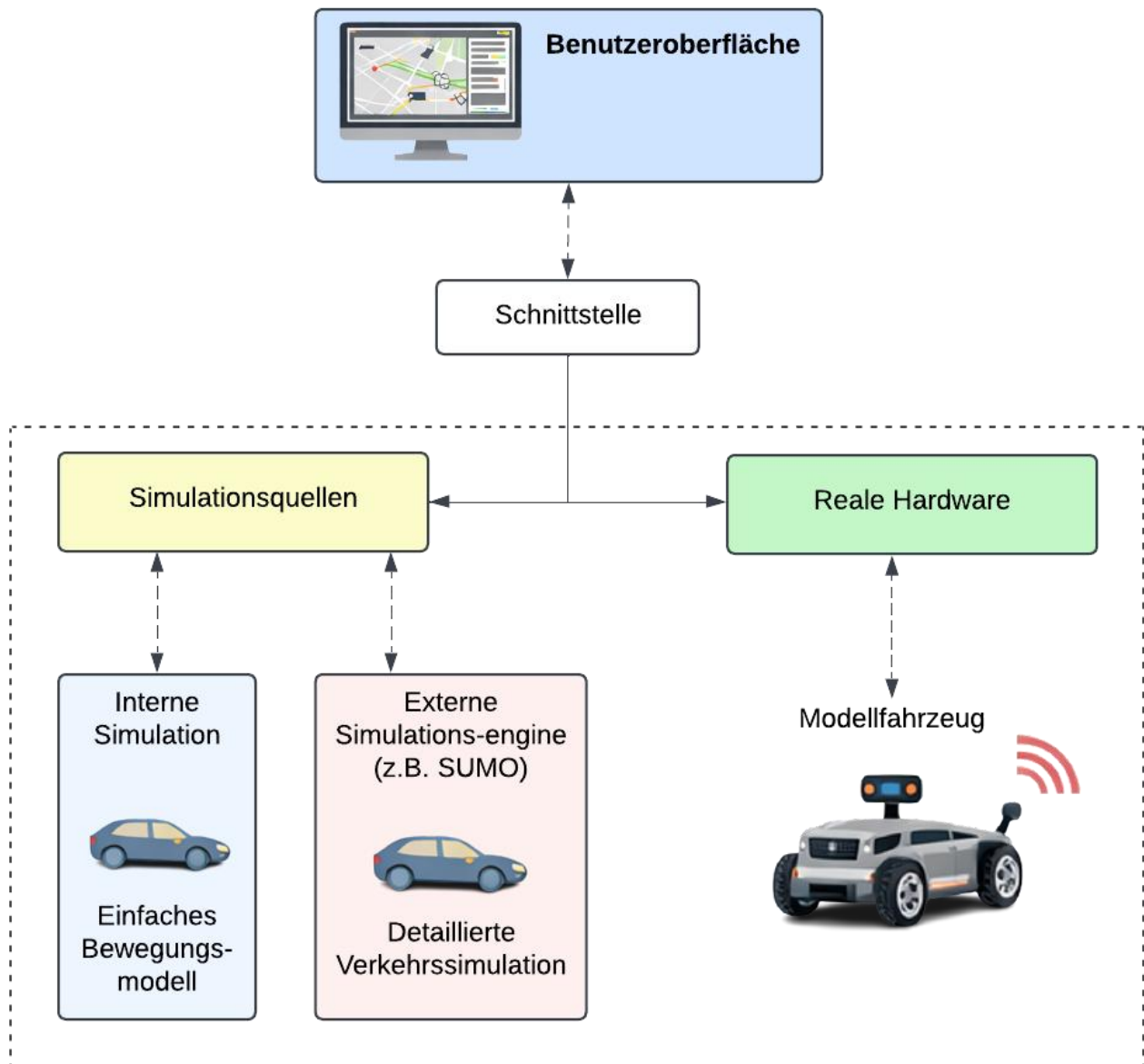


Abbildung 1: Hybride Architektur der Verkehrssimulation

## 2.4 Webtechnologien für Visualisierung und Interaktion

Die Benutzeroberfläche wird auf Basis moderner Webtechnologien entwickelt. Ziel ist es, eine interaktive, plattformunabhängige und leicht zugängliche Benutzeroberfläche bereitzustellen. Webbasierte Anwendungen können direkt im Webbrowser genutzt werden, ohne dass eine lokale Installation erforderlich ist. Dies vereinfacht die Nutzung und ist insbesondere für den Einsatz in Lehr- und Laborumgebungen von Vorteil.

Für die Entwicklung der Benutzeroberfläche kommen verschiedene Technologien zum Einsatz, die im Folgenden vorgestellt werden.

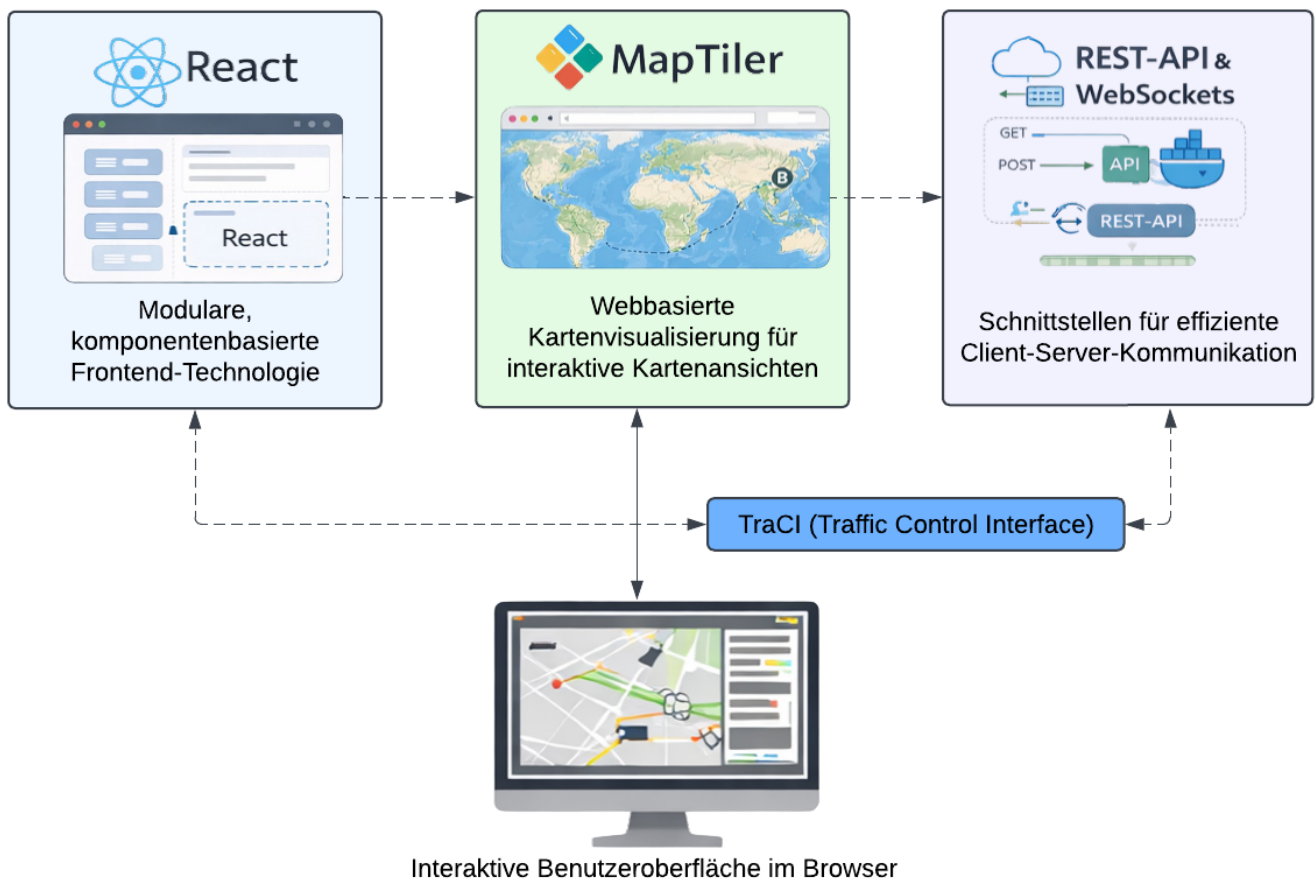


Abbildung 2: Webtechnologien für Visualisierung und Interaktion

### 2.4.1 React

Für die Entwicklung der Benutzeroberfläche wird die JavaScript-Bibliothek React verwendet. React basiert auf einem komponentenorientierten Ansatz. Die Benutzeroberfläche wird dabei in einzelne, wiederverwendbare Bausteine unterteilt, wie beispielsweise Kartenansicht, Werkzeugleiste oder Fahrzeugdialog.

Ein wesentlicher Vorteil von React besteht darin, dass Änderungen im Zustand der Anwendung automatisch zu einer Aktualisierung der Darstellung im Browser führen. Dies ist besonders wichtig für interaktive Simulationen, da sich Fahrzeugpositionen und Systemzustände fortlaufend ändern.

Durch die modulare Struktur eignet sich React gut für komplexe Anwendungen mit klar getrennten Aufgabenbereichen. In der vorliegenden Arbeit ermöglicht dies eine saubere Trennung zwischen Benutzeroberfläche, Simulationslogik und Datenverwaltung.

22

### 2.4.2 MapTiler / Kartenvisualisierung

Für die Darstellung des Straßennetzes wird ein webbasiertes Kartensystem auf Basis von MapTiler verwendet. MapTiler stellt Kartendaten und Kartenkacheln bereit, die im Browser visualisiert werden können.

Die Kartenansicht bildet die zentrale Komponente zur Darstellung der Simulationsumgebung. Hier werden Straßen, Fahrzeuge und weitere Verkehrselemente dargestellt. Die Nutzung einer webbasierten Kartenlösung ermöglicht eine flexible Integration von OpenStreetMap-Daten sowie eine interaktive Darstellung von Objekten.

Ein Vorteil der webbasierten Kartenvisualisierung liegt in der direkten Verbindung zur Benutzerinteraktion. Fahrzeuge können positioniert, Routen gesetzt oder Verkehrssituationen unmittelbar dargestellt werden. Dadurch entsteht eine intuitive Arbeitsumgebung für Experimente mit Verkehrssimulationen.

23

---

<sup>22</sup> React. (kein Datum). Eine JavaScript Bibliothek zum Erstellen von Benutzeroberflächen.

<sup>23</sup> maptiler. (kein Datum). Maps for developers.

### 2.4.3 REST-APIs & WebSockets

Zur Kommunikation zwischen Frontend und Backend werden REST-Schnittstellen und WebSockets eingesetzt. REST-APIs basieren auf dem HTTP-Protokoll und eignen sich besonders für einzelne Steuer- und Konfigurationsoperationen, wie beispielsweise das Starten oder Stoppen einer Simulation, das Hinzufügen von Fahrzeugen oder das Anpassen von Szenarien. WebSockets hingegen ermöglichen eine dauerhafte Verbindung zwischen Client und Server, über die Daten in beide Richtungen übertragen werden können. Sie werden verwendet, um Simulationsdaten in Echtzeit zu übertragen, beispielsweise Fahrzeugpositionen oder Statusinformationen. Dadurch kann die Benutzeroberfläche kontinuierlich aktualisiert werden, ohne dass ständig neue Anfragen an den Server gesendet werden müssen. Die Kombination aus REST und WebSockets ermöglicht eine effiziente und flexible Kommunikation innerhalb der Simulationsplattform. Während REST hauptsächlich für Steuerbefehle genutzt wird, übernimmt WebSocket die kontinuierliche Übertragung aktueller Simulationsdaten.<sup>24</sup>

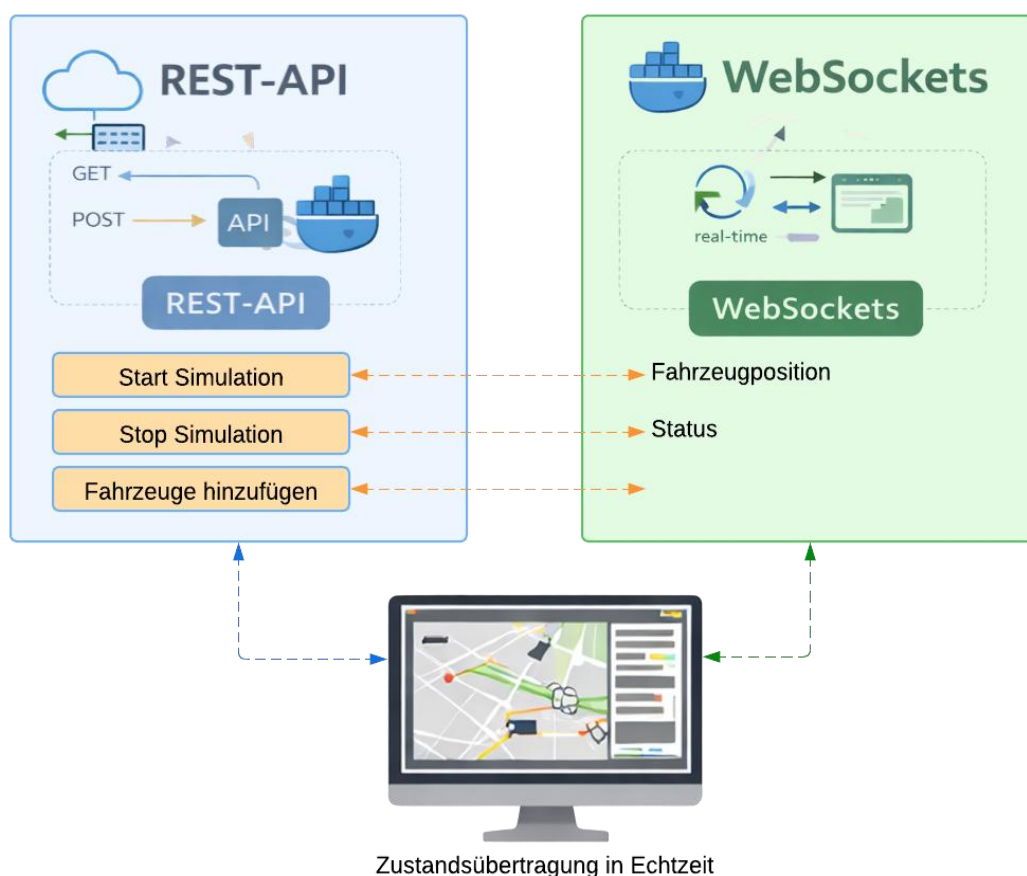


Abbildung 3: REST APIs & WebSockets

<sup>24</sup> Svenja Hilbrich. (kein Datum). Entwicklung webbasierter graphischer Benutzeroberflächen für Simulationskerne.

### 3. Methodisches Vorgehen

Diese Arbeit folgt einem gestaltungsorientierten Forschungsansatz und orientiert sich am Rahmenwerk des Design Science Research (DSR) nach Hevner et al. (2004). DSR ist ein etablierter Forschungsansatz in der Informatik und der angewandten Wissenschaft, der auf die Entwicklung und Evaluation von Artefakten ausgerichtet ist. Als Artefakt wird dabei jedes künstlich geschaffene Objekt verstanden, das ein bestehendes Problem löst – in dieser Arbeit eine webbasierte Benutzeroberfläche für SUMO, die nicht-spezialisierte Nutzer in Lehr- und Laborumgebungen unterstützt.

DSR eignet sich besonders für diese Arbeit, weil das Ziel nicht die Generierung rein theoretischer Erkenntnisse ist, sondern die Entwicklung einer praktischen Lösung für ein klar identifiziertes Problem: Die bestehende SUMO-GUI ist für nicht-spezialisierte Nutzer zu komplex. DSR legitimiert diesen Ansatz wissenschaftlich, indem es die Entwicklung des Artefakts als eigenständigen Forschungsbeitrag anerkennt, sofern die Lösung systematisch begründet, entwickelt und evaluiert wird. Das DSR-Rahmenwerk gliedert sich in drei zentrale Phasen, die wie folgt auf diese Arbeit angewendet werden:

*Tabelle 1: Anwendung des DSR-Rahmenwerks auf die vorliegende Arbeit*

| DSR-Phase                        | Umsetzung in dieser Arbeit                                                             |
|----------------------------------|----------------------------------------------------------------------------------------|
| <b>1. Problem identifizieren</b> | Kap. 1.2: SUMO ist für Lehr- und Laborumgebungen zu komplex und schwer zugänglich      |
| <b>2. Artefakt entwickeln</b>    | Kap. 5–6: Konzeption und Implementierung der webbasierten GUI mit hybrider Architektur |
| <b>3. Artefakt evaluieren</b>    | Kap. 7: Funktionale Tests und Vergleich mit der SUMO-GUI                               |

25

---

<sup>25</sup> Hevner, A. R. (2004). Design Science in Information Systems Research.

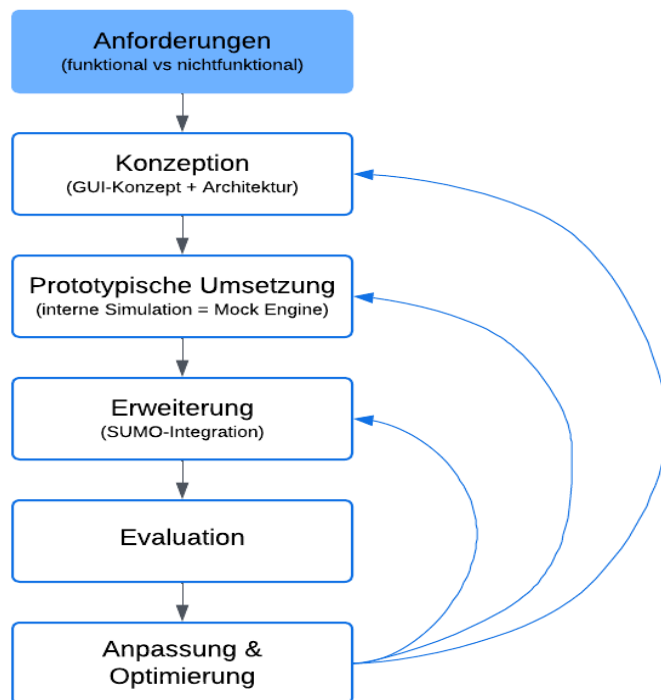


Abbildung 4: Iterativer Entwicklungsprozess

### Iterativer Entwicklungsprozess

Die Umsetzung erfolgt in einem iterativen Entwicklungsprozess. Zunächst werden auf Basis der identifizierten Problemstellung funktionale und nichtfunktionale Anforderungen erhoben (Kapitel 4). Darauf aufbauend wird die Systemarchitektur konzipiert (Kapitel 5) und prototypisch implementiert (Kapitel 6). Jede Entwicklungsphase wird durch Tests begleitet, um sicherzustellen, dass die Kernfunktionen korrekt umgesetzt werden. Anforderungen an Bedienung, Visualisierung und Echtzeitverhalten werden dabei während der Entwicklung bei Bedarf angepasst. Abbildung 4 visualisiert diesen iterativen Prozess.

### Evaluationsstrategie

Die Evaluation des entwickelten Artefakts erfolgt in Kapitel 7 in zwei Schritten: Erstens werden funktionale Tests der zentralen Systemfunktionen durchgeführt, die systematisch überprüfen, ob die in Kapitel 4 definierten Anforderungen erfüllt wurden. Zweitens wird ein qualitativer Vergleich mit der bestehenden SUMO-GUI anhand definierter Kriterien wie Einstiegshürde, Interaktivität und Eignung für Lehr- und Laborumgebungen durchgeführt. Diese Kombination ermöglicht eine mehrdimensionale Bewertung des Artefakts hinsichtlich Funktionalität und Bedienbarkeit.



## 4. Analyse der Anforderungen

Die Entwicklung einer webbasierten hybriden Verkehrssimulationsumgebung erfordert eine systematische Analyse der Anforderungen an das System. Diese Anforderungen ergeben sich aus der Zielsetzung der Arbeit, der geplanten Nutzung in Lehr- und Laborumgebungen sowie aus der geplanten Erweiterbarkeit der Plattform.

Im Folgenden werden funktionale und nichtfunktionale Anforderungen unterschieden. Funktionale Anforderungen beschreiben die konkreten Funktionen und Aufgaben des Systems. Nichtfunktionale Anforderungen definieren wichtige Eigenschaften des Systems, wie beispielsweise Benutzerfreundlichkeit, Erweiterbarkeit und technische Stabilität.

### 4.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben die Funktionen, die die entwickelte Simulationsplattform aus Sicht der Nutzer bereitstellen muss. Sie legen fest, welche Interaktionen, Steuerungsmöglichkeiten und Abläufe das System unterstützen soll.

Die funktionalen Anforderungen der entwickelten Verkehrssimulationsplattform sind in Tabelle 4.1 strukturiert dargestellt.

*Tabelle 2: Funktionale Anforderungen (Tabellenform):*

| Nr | Kategorie      | Anforderung & Beschreibung                                                                                                      | Priorität | Akzeptanzkriterium                                                                        |
|----|----------------|---------------------------------------------------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------|
| 1  | Visualisierung | <b>Straßennetz darstellen</b><br>Straßen, Kreuzungen und Infrastruktur werden in einer interaktiven Kartenansicht visualisiert. | Muss      | Karte lädt, Zoom/Pan möglich, Netz ist sichtbar.                                          |
| 2  | Interaktion    | Nutzer können Fahrzeuge interaktiv in der Karte platzieren und Startposition setzen.                                            | Muss      | Klick/Interaktion erzeugt Fahrzeug an Position; Fahrzeug erscheint sofort in der Ansicht. |

|   |                      |                                                                                                                                           |        |                                                                                             |
|---|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------|
| 3 | Konfiguration        | <b>Fahrzeugtypen auswählen</b><br>Auswahl unterschiedlicher Fahrzeugtypen (PKW, LKW, Bus, Motorrad).                                      | Soll   | Mind. 2 Fahrzeugtypen verfügbar.                                                            |
| 4 | Verwaltung           | <b>Mehrere Fahrzeuge verwalten</b><br>Mehrere Fahrzeuge sind gleichzeitig verwaltbar (Liste/Übersicht), identifizierbar und selektierbar. | Muss   | Fahrzeugliste zeigt alle Fahrzeuge; Auswahl hebt Fahrzeug hervor; eindeutige IDs vorhanden. |
| 5 | Steuerung            | <b>Simulation starten</b><br>Start der Simulation über die GUI.                                                                           | Muss   | Start-Button setzt Status auf „laufend“; Positionsupdates beginnen.                         |
| 6 | Steuerung            | <b>Simulation pausieren</b><br>Pausieren stoppt den Fortschritt, Zustand bleibt erhalten.                                                 | Muss   | Pause stoppt Positionsänderungen; nach Fortsetzen geht es weiter ohne Reset.                |
| 7 | Steuerung            | <b>Simulationsgeschwindigkeit anpassen</b><br>Nutzer kann die Geschwindigkeit der Simulation verändern.                                   | Muss   | Änderung wirkt sich sichtbar auf Update-Rate/Simulationszeit aus.                           |
| 8 | Hybrid-fähigkeit     | <b>Interne + externe Engine nutzbar</b><br>System unterstützt interne Simulation und externe Engine (SUMO).                               | Muss   | Wechsel zwischen interner Engine und SUMO möglich (mind. im Backend/Prototyp).              |
| 9 | Hardware-Perspektive | <b>Schnittstellen für reale Komponenten</b><br>Architektur sieht Schnittstellen vor, um später reale Fahrzeuge/Roboter anzubinden.        | Option | Konzept/Architektur zeigt klaren Erweiterungspunkt; Interface ist im Design vorgesehen.     |

## 4.2 Nichtfunktionale Anforderungen

Neben den funktionalen Anforderungen sind auch qualitative Eigenschaften entscheidend für die Nutzbarkeit und Erweiterbarkeit des Systems. Nichtfunktionale Anforderungen beschreiben wichtige Eigenschaften der Simulationsplattform. Sie beziehen sich nicht auf konkrete Funktionen, sondern auf Aspekte wie Benutzerfreundlichkeit, Leistungsfähigkeit, Erweiterbarkeit und technische Stabilität. Die identifizierten nichtfunktionalen Anforderungen sind in Tabelle 4.2 strukturiert dargestellt.

*Tabelle 3: Nichtfunktionale Anforderungen (Tabellenform):*

| Nr | Qualitätsmerkmal  | Ziel / Kriterium                                                                                                          | Nachweis / Messung                                                                                                                                |
|----|-------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Usability         | <b>Intuitive Bedienung</b><br>Kern-Use-Cases ohne Anleitung durchführbar (Fahrzeug platzieren, Start/Pause).              | Explorativer Usability-Test mit Kommilitonen – alle Kernaufgaben wurden ohne Anleitung in 1–2 Minuten erfolgreich abgeschlossen (siehe Kap. 7.2). |
| 2  | Performance       | <b>Reaktionszeit UI</b><br>UI-Reaktion auf Aktionen (z. B. Klick, Button) ohne spürbare Verzögerung.                      | Praktischer Test: Aktion → sichtbares Feedback unmittelbar. Kein messbarer Lag bei Start, Pause und Fahrzeugplatzierung beobachtet.               |
| 3  | Echtzeitfähigkeit | <b>Kontinuierliche Updates</b><br>Fahrzeugzustände werden regelmäßig übertragen und visualisiert (via WebSocket).         | Beobachtung: Positionsupdates laufen kontinuierlich während der Simulation ohne Unterbrechung (siehe Kap. 7.1).                                   |
| 4  | Erweiterbarkeit   | <b>Neue Module integrierbar</b><br>System erlaubt Erweiterungen (weitere Engines/Module) ohne Umbau des Gesamtsystems.    | Abstrakte Engine-Schnittstelle im Backend nachgewiesen; SUMO und interne Simulation implementieren dieselbe Schnittstelle (siehe Kap. 6.2.3).     |
| 5  | Modularität       | <b>Klare Trennung der Komponenten</b><br>Frontend enthält keine Simulationslogik; Backend kapselt Simulation und Engines. | Architekturdiagramm und Code-Struktur belegen die Trennung von Frontend, Backend und Simulationsengines (siehe Kap. 5 und 6).                     |

|   |                                |                                                                                                                               |                                                                                                                                                                           |
|---|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6 | Wartbarkeit/<br>Testbarkeit    | <b>Komponenten isoliert<br/>prüfbar</b><br>Simulation kann<br>unabhängig vom UI getestet<br>werden (z. B. interne<br>Engine). | Funktionale Tests in Kap. 7.1<br>wurden ohne laufendes<br>Frontend durchgeführt;<br>getrennte Module ermöglichen<br>isolierte Prüfung.                                    |
| 7 | Reproduzierbarkeit<br>(Docker) | <b>Einheitliche<br/>Laufzeitumgebung</b><br>System ist containerisiert<br>ausführbar, inkl. SUMO in<br>separatem Container.   | docker-compose startet das<br>Gesamtsystem auf einer neuen<br>Maschine vollständig; alle<br>Container (Frontend, Backend,<br>SUMO, DB) laufen stabil<br>(siehe Kap. 6.4). |

### 4.3 Nutzerrollen & Anwendungskontext

Die entwickelte Simulationsumgebung richtet sich an unterschiedliche Nutzergruppen mit jeweils eigenen Anforderungen an Bedienbarkeit, Erweiterbarkeit und technische Integration. Die identifizierten Nutzerrollen sowie deren typische Nutzungsszenarien werden in der Abbildung „Nutzerrollen & Anwendungskontext“ zusammenfassend dargestellt.

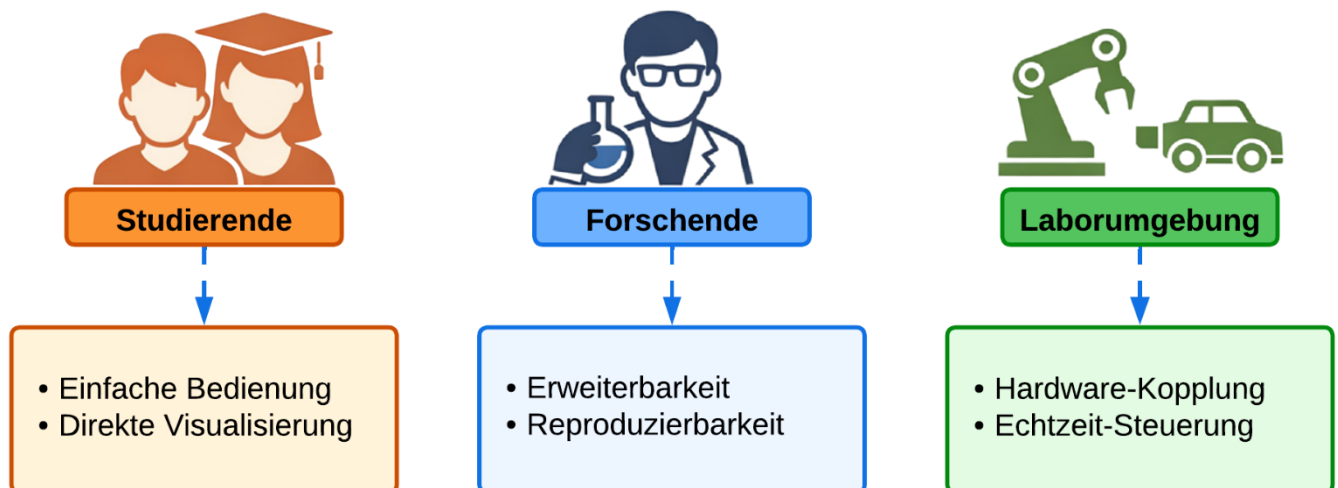


Abbildung 5: Nutzerrollen & Anwendungskontext

## 5. Konzeption und Systemarchitektur

Dieses Kapitel beschreibt die Konzeption der webbasierten Benutzeroberfläche sowie die technische Systemarchitektur der Verkehrssimulationsplattform. Beide Bereiche sind eng miteinander verzahnt: Die Anforderungen an die Oberfläche bestimmen maßgeblich den Aufbau des Gesamtsystems, während die Architektur die technischen Rahmenbedingungen für die Umsetzung der Benutzeroberfläche vorgibt. Im ersten Teil werden die Ziele und das Layout der GUI beschrieben sowie die zentralen Oberflächenkomponenten vorgestellt. Im zweiten Teil wird die daraus abgeleitete Systemarchitektur mit ihren Schichten, Schnittstellen und der containerisierten Infrastruktur erläutert.

### 5.1 Ziele der GUI

Die Benutzeroberfläche bildet die zentrale Oberfläche zur Interaktion mit der entwickelten Verkehrssimulationsplattform. Sie ermöglicht die visuelle Darstellung des Straßennetzes, die Erstellung und Konfiguration von Verkehrsszenarien sowie die Steuerung und Beobachtung des Simulationsverlaufs. Ziel ist die Entwicklung einer übersichtlichen, modular aufgebauten und intuitiv bedienbaren Benutzeroberfläche, die sowohl für virtuelle Simulationen als auch für zukünftige hybride Laboranwendungen geeignet ist.

Die Konzeption der Benutzeroberfläche verfolgt mehrere zentrale Ziele. Erstens soll sie eine direkte und visuelle Interaktion mit dem Simulationsmodell ermöglichen. Nutzer sollen zentrale Aktionen wie das Platzieren von Fahrzeugen, das Starten der Simulation oder das Anpassen von Parametern direkt über die Oberfläche durchführen können. Zweitens soll die GUI eine transparente Darstellung des aktuellen Simulationszustands bereitstellen – relevante Informationen wie Fahrzeugpositionen, aktive Szenarien oder der Status der Simulation müssen jederzeit nachvollziehbar sein.

Drittens soll der Simulationsablauf konsistent und leicht verständlich steuerbar sein. Zentrale Funktionen wie Start, Pause, Reset oder die Anpassung der Simulationsgeschwindigkeit sollen klar erkennbar und ohne Umwege erreichbar sein. Ein weiteres Ziel besteht in der Engine-Unabhängigkeit der Benutzeroberfläche: Die GUI soll unabhängig davon funktionieren, ob eine interne Simulation oder eine externe Simulationsengine wie SUMO verwendet wird. Schließlich soll die Benutzeroberfläche erweiterbar bleiben, sodass zusätzliche Funktionen oder zukünftig reale Hardwarekomponenten ohne Änderungen an der Grundstruktur integriert werden können.

## 5.2 Layout der Benutzeroberfläche

Die Benutzeroberfläche folgt einem klar strukturierten Layout, das die zentralen Funktionsbereiche voneinander trennt. Im Zentrum der Anwendung befindet sich die Kartenansicht, die das Straßennetz, Fahrzeugpositionen und weitere Verkehrselemente darstellt und zugleich als zentrale Arbeitsfläche für direkte Interaktionen dient. Ergänzt wird sie durch eine dauerhaft sichtbare Werkzeugleiste mit Funktionen wie Start, Pause, Reset und Geschwindigkeitsanpassung. Ein seitlicher Bereich zur Fahrzeugverwaltung ermöglicht die übersichtliche Organisation mehrerer aktiver Fahrzeuge. Für die Eingabe spezifischer Parameter werden kontextabhängige Dialogfenster eingesetzt, die nur bei Bedarf erscheinen und die Hauptoberfläche im normalen Betrieb übersichtlich halten. Insgesamt folgt das Layout dem Prinzip, häufig genutzte Funktionen direkt sichtbar zu machen und seltenere Konfigurationsschritte in separate Dialoge auszulagern.



Abbildung 6: Benutzeroberfläche der GUI



## 5.3 Zentrale GUI-Komponenten

### 5.3.1 Werkzeugleiste



Abbildung 7: Werkzeugleiste (Steuerung der Simulation)

Die Werkzeugleiste dient der zentralen Steuerung der Simulation und ist dauerhaft sichtbar, um einen schnellen Zugriff auf zentrale Funktionen zu gewährleisten. Sie enthält Bedienelemente für:

- Start und Pause der Simulation
- Zurücksetzen des Systems
- Anpassung der Simulationsgeschwindigkeit
- Aktivierung bestimmter Modi (z. B. Platzierungsmodus)

Ziel ist eine übersichtliche und reduzierte Gestaltung für den Nutzer

### 5.3.2 Kartenansicht

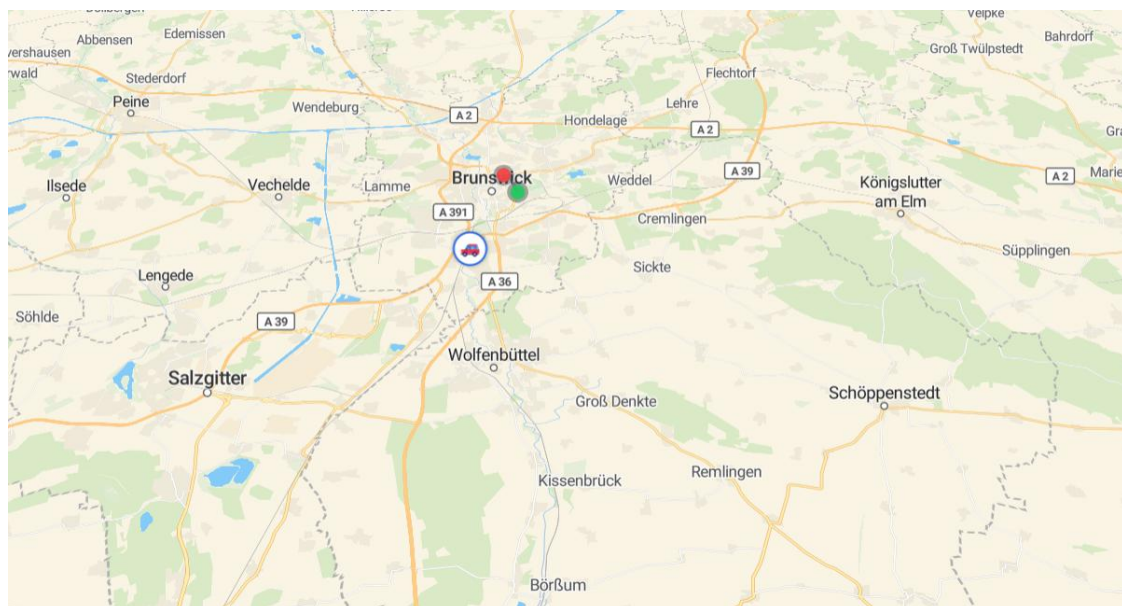
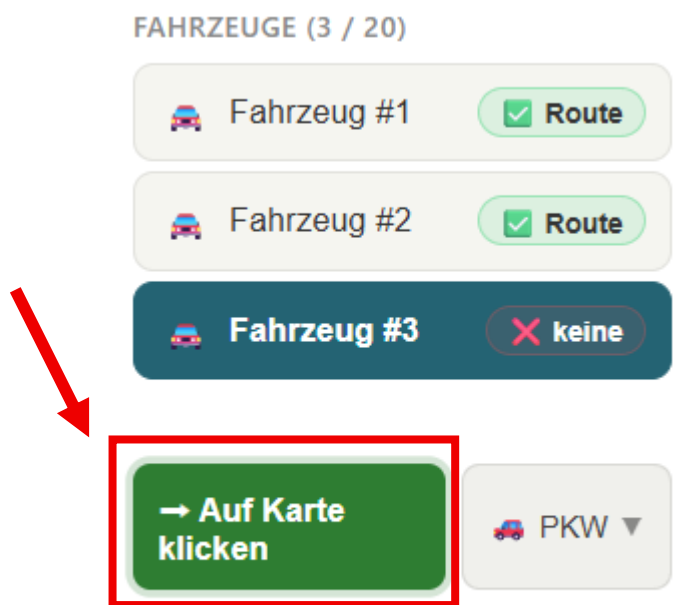


Abbildung 8: Kartenansicht (MapTiler)

Die Kartenansicht bildet das visuelle Zentrum der Anwendung. Sie dient zur Darstellung von Straßen und Kreuzungen, Fahrzeugpositionen, Routen sowie Signalzuständen. Alle Interaktionen – wie das Platzieren von Fahrzeugen oder das Festlegen von Start- und Zielpunkten – erfolgen direkt in der Kartenansicht. Dadurch entsteht eine unmittelbare Verbindung zwischen Benutzeraktion und visueller Rückmeldung. Die Kartenansicht übernimmt somit sowohl Visualisierungs- als auch Interaktionsaufgaben.

### 5.3.3 Fahrzeugmanagement

Das Fahrzeugmanagement ermöglicht die Erstellung, Auswahl und Verwaltung einzelner Fahrzeuge innerhalb der Simulation. Nutzer können neue Fahrzeuge hinzufügen, bestehende auswählen, Routen zuweisen und Fahrzeuge entfernen. Jedes Fahrzeug wird als eigenständiges Objekt mit spezifischen Eigenschaften verwaltet. Die Verwaltung erfolgt über eine seitliche Übersicht, wodurch eine strukturierte Organisation mehrerer Fahrzeuge möglich ist.



ROUTE FÜR AUSGEWÄHLTES  
FAHRZEUG

Abbildung 9: Fahrzeugmanagement



### 5.3.4 Dialogsystem

Zur Konfiguration spezifischer Parameter werden kontextabhängige Dialogfenster eingesetzt. Diese ermöglichen beispielsweise die Auswahl eines Fahrzeugtyps, die Festlegung von Start- und Zielpunkten sowie die Anpassung von Simulationsparametern. Das Dialogsystem wird nur dann angezeigt, wenn eine entsprechende Aktion ausgelöst wird. Dadurch bleibt die Hauptoberfläche übersichtlich.

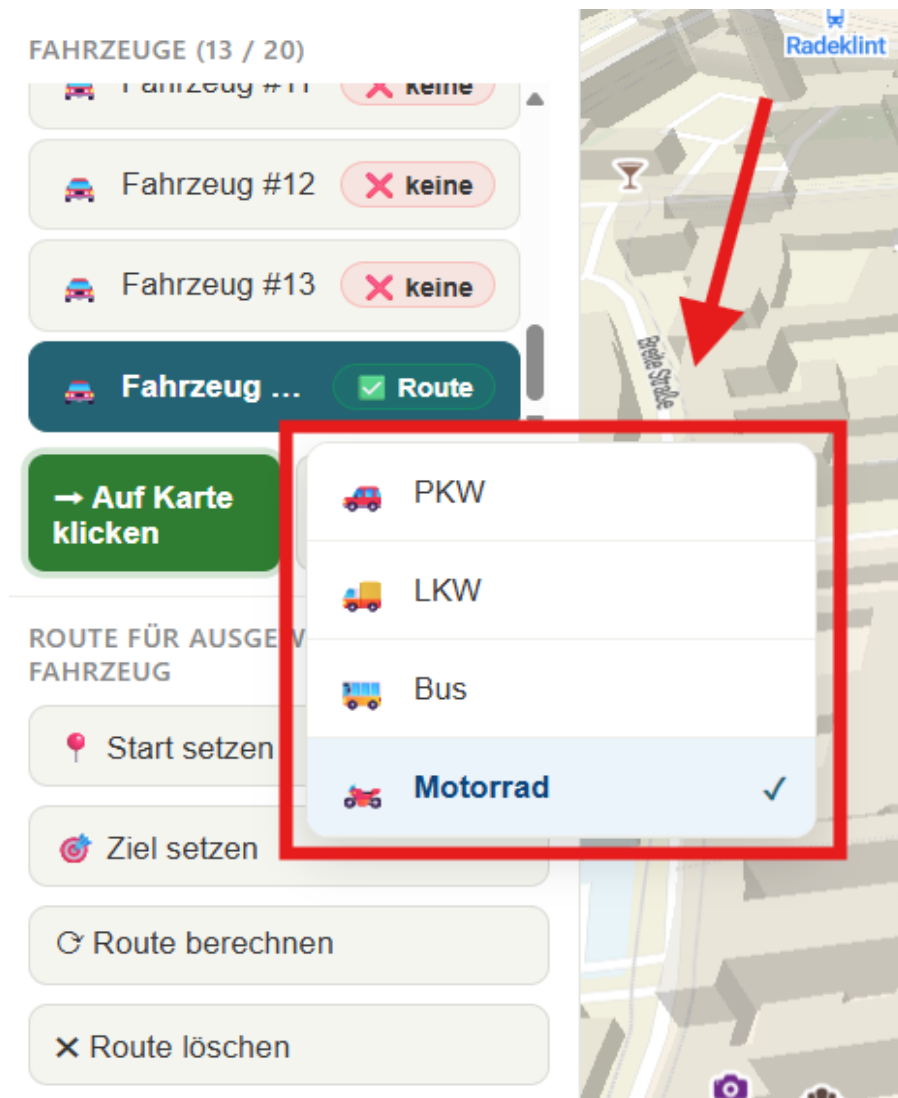


Abbildung 10: Dialogsystem (Fahrzeugtypen)

## 5.4 Systemarchitektur

Die technische Systemarchitektur der Plattform wurde aus den zuvor beschriebenen Anforderungen an die Benutzeroberfläche abgeleitet. Sie basiert auf einer klaren Trennung zwischen Benutzeroberfläche, serverseitiger Anwendungslogik und Simulationskomponenten. Dadurch entsteht eine modulare Systemstruktur, die leicht erweitert werden kann und sowohl eine interne Simulation als auch externe Simulationsengines integrieren kann. Die wichtigsten Bestandteile der Plattform sind das Web-Frontend zur Visualisierung und Interaktion, das Backend zur Steuerung der Systemlogik, unterschiedliche Simulationsengines zur Berechnung der Verkehrsdynamik sowie eine Datenhaltung zur Speicherung von Szenarien und Konfigurationen.

### 5.4.1 Frontend

Das Frontend stellt die Benutzeroberfläche der Plattform dar und wird im Webbrowser ausgeführt. Es ermöglicht die visuelle Darstellung des Straßennetzes sowie die Interaktion mit der Simulation. Das Frontend enthält selbst keine komplexe Simulationslogik, sondern dient als Interaktions- und Visualisierungsschicht. Die Kommunikation mit dem Backend erfolgt über REST-basierte Schnittstellen sowie über WebSocket-Verbindungen für Echtzeitdaten.

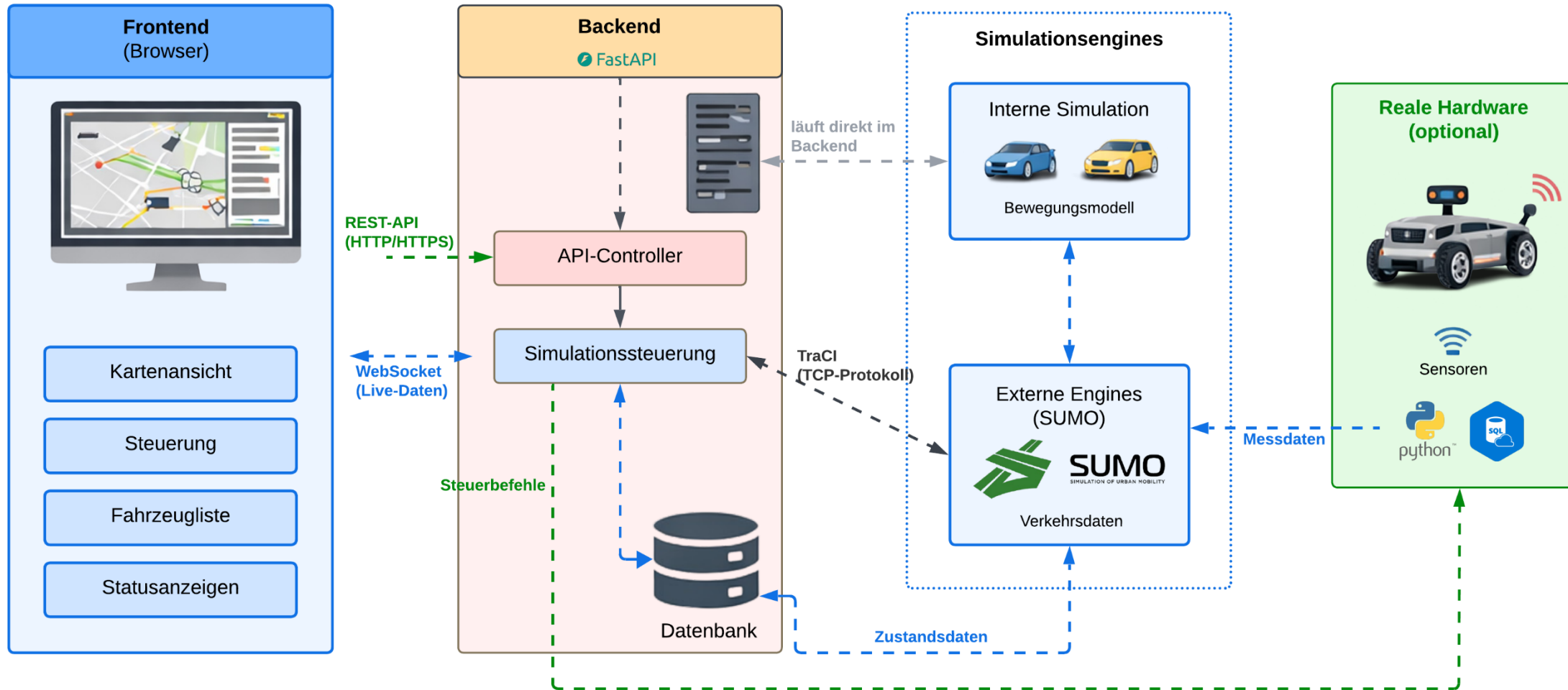


Abbildung 11: Gesamtarchitektur der Verkehrssimulationsplattform

### 5.4.2 Backend (FastAPI)

Das Backend übernimmt die zentrale Koordination der Plattform. Es verarbeitet Benutzeranfragen, verwaltet den globalen Simulationszustand und kommuniziert mit den Simulationsengines. Zu den zentralen Aufgaben gehören die Bereitstellung einer API für das Frontend, die Verwaltung von Fahrzeugen, Szenarien und Simulationsparametern, die Steuerung des Simulationsablaufs sowie die Integration interner und externer Simulationsengines. Durch diese Struktur bleibt das Frontend von der konkreten Implementierung der Simulationslogik entkoppelt, sodass unterschiedliche Simulationsquellen integriert werden können, ohne Änderungen an der Benutzeroberfläche vorzunehmen.

### 5.4.3 Simulationsengines

Die Plattform unterstützt zwei Varianten von Simulationsengines, die über eine abstrakte Schnittstelle in das Backend integriert werden:

- Interne Simulation (Mock Engine): Eine vereinfachte interne Simulation dient als Referenzimplementierung für prototypische Tests und Demonstrationszwecke. Sie ermöglicht grundlegende Verkehrsbewegungen ohne externe Abhängigkeiten.
- Externe Simulation (SUMO): SUMO übernimmt die detaillierte mikroskopische Simulation komplexer Verkehrsnetze und stellt Fahrzeugzustände für die Visualisierung bereit.

### 5.4.4 Datenbank

Die Datenbank dient der persistenten Speicherung von Szenarien, Fahrzeugkonfigurationen, Simulationseinstellungen und optionalen Analyseergebnissen. Durch die Speicherung von Szenarien können Experimente reproduzierbar durchgeführt werden. Die Datenbank ist architektonisch vom Backend getrennt und wird ausschließlich über definierte Schnittstellen angesprochen.

## 5.5 Kommunikation zwischen Frontend und Backend

Das Frontend dient als Interaktionsschicht, über die Nutzer Eingaben vornehmen. Diese Aktionen werden an das Backend weitergeleitet, wo sie verarbeitet und in den globalen Simulationszustand übernommen werden. Für einzelne Steuerbefehle – wie das Starten und Stoppen der Simulation, das Hinzufügen oder Entfernen von Fahrzeugen oder das Anpassen von Parametern – wird eine REST-basierte Kommunikation verwendet. Simulationsdaten wie Fahrzeugpositionen und Systemzustände werden hingegen über WebSocket-Verbindungen kontinuierlich in Echtzeit an das Frontend übertragen. Ein wesentliches Merkmal dieser Kommunikationsstruktur ist die klare Trennung der Zuständigkeiten: Das Frontend ist ausschließlich für Darstellung und Interaktion verantwortlich, während das Backend die gesamte Systemlogik übernimmt.

## 5.6 Struktur der Simulationsdaten

Die Simulationsplattform verarbeitet verschiedene Arten von Daten, die in einer klaren Struktur organisiert sind. Zu den zentralen Datenkategorien gehören Fahrzeuge, Straßen-

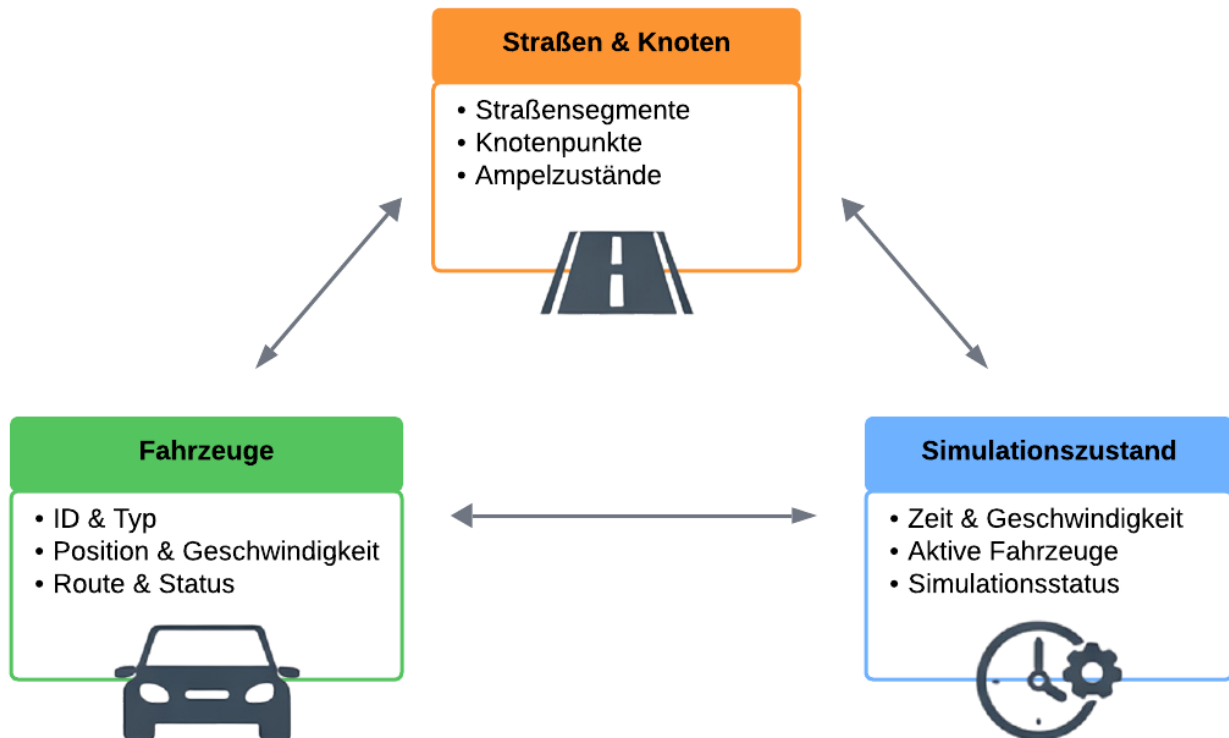


Abbildung 12: Datenmodell

und Infrastrukturinformationen sowie der globale Simulationszustand. Ein Fahrzeug wird als eigenständiges Objekt mit einer eindeutigen Identifikationsnummer, der aktuellen Position, der Geschwindigkeit, der geplanten Route sowie einem Statusattribut verwaltet. Das Straßennetz besteht aus Straßenabschnitten, Knotenpunkten und Infrastrukturelementen wie Lichtsignalanlagen. Der globale Simulationszustand enthält Informationen zur aktuellen Simulationszeit, der eingestellten Geschwindigkeit, dem Simulationsstatus sowie der Liste aller aktiven Fahrzeuge. Die Datenstruktur ist so gestaltet, dass sie unabhängig von der verwendeten Simulationsengine funktioniert – sowohl die interne Simulation als auch SUMO können auf dieselbe Struktur zugreifen.

## 5.7 Docker-basierte Infrastruktur

Zur Sicherstellung einer reproduzierbaren und plattformunabhängigen Laufzeitumgebung wird die Simulationsplattform containerbasiert betrieben. Der Einsatz von Docker ermöglicht eine klare Definition aller Software-Abhängigkeiten, konsistente Laufzeitumgebungen sowie eine einfache Bereitstellung auf unterschiedlichen Systemen. In der entwickelten Plattform werden folgende Komponenten containerisiert:

- Frontend
- Backend-Server
- Externe Simulationsengine (SUMO)
- Optionale Datenbank

Durch diese Struktur kann das Gesamtsystem mit wenigen Befehlen gestartet werden. Dies

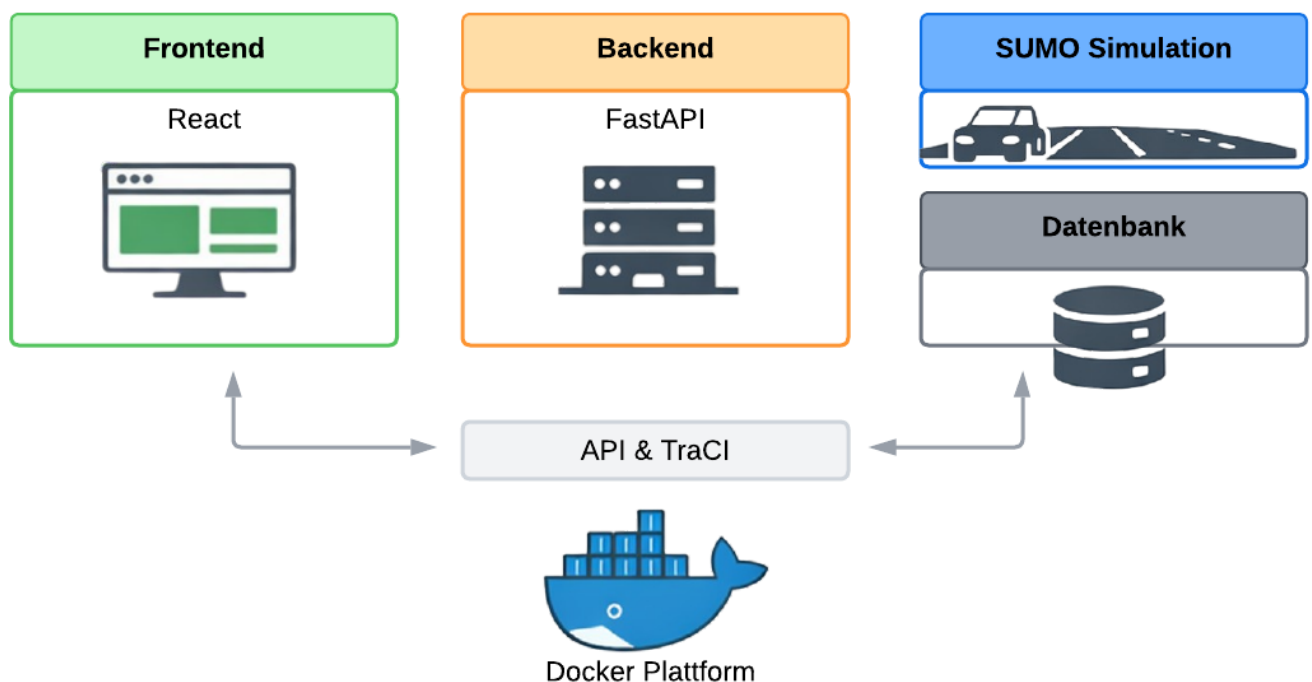


Abbildung 13: Container Architektur

erleichtert insbesondere den Einsatz in Lehrveranstaltungen oder Laborumgebungen, da komplexe Installationsprozesse vermieden werden. Darüber hinaus unterstützt die containerbasierte Infrastruktur die Reproduzierbarkeit von Experimenten, da Simulationen unter identischen technischen Bedingungen erneut ausgeführt werden können.

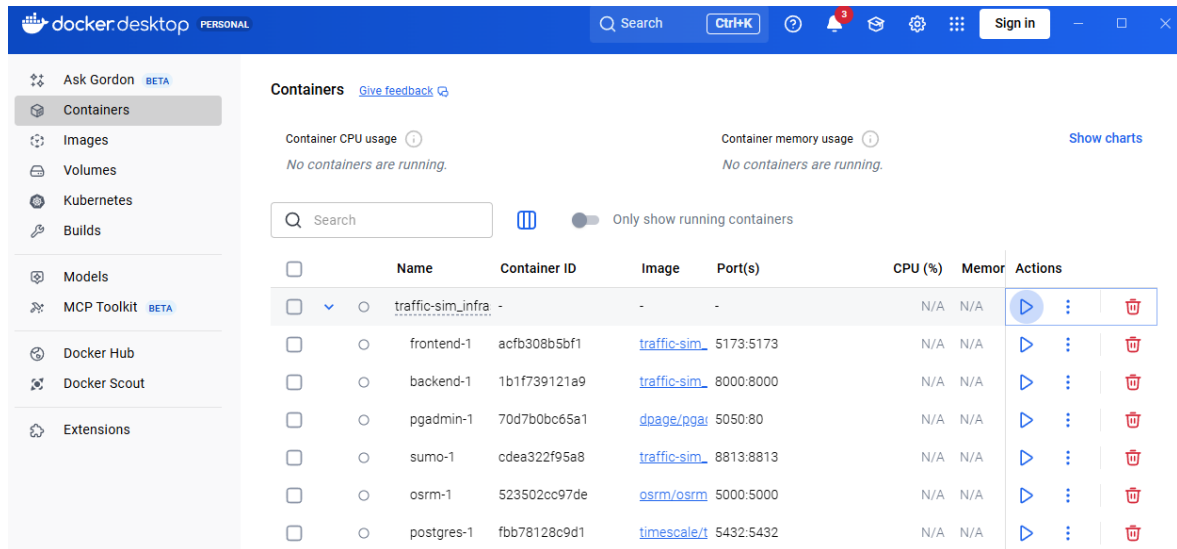


Abbildung 14: Docker (Container)



## 6. Implementierung

In diesem Kapitel wird die technische Umsetzung der entwickelten webbasierten Verkehrssimulationsplattform beschrieben. Im Fokus stehen die Implementierung des Frontends, des Backends sowie der internen Simulationslogik und die Integration der externen Simulationsengine SUMO. Ziel war die Entwicklung eines funktionsfähigen Prototyps, der die interaktive Konfiguration, Steuerung und Visualisierung von Verkehrsszenarien in Echtzeit ermöglicht. Dabei wurde besonderer Wert auf eine modulare und erweiterbare Struktur gelegt, die beide Simulationsvarianten – interne Simulation und SUMO – über eine einheitliche Benutzeroberfläche zugänglich macht.

### 6.1 Implementierung des Frontends

Das Frontend der Verkehrssimulationsplattform wurde unter Verwendung der JavaScript-Bibliothek React umgesetzt. Ziel war die Entwicklung einer interaktiven, modularen und responsiven Benutzeroberfläche, die die in Kapitel 5 definierten Anforderungen erfüllt. Das Frontend übernimmt ausschließlich die Visualisierung und Benutzerinteraktion – die Simulation selbst sowie die Verwaltung des globalen Zustands werden im Backend durchgeführt. Diese Trennung erhöht die Modularität und erleichtert die Erweiterbarkeit des Systems.

Die Kommunikation zwischen Frontend und Backend erfolgt über:

- REST-Schnittstellen für Steuerbefehle (z. B. Start, Pause, Reset, Fahrzeug hinzufügen)
- WebSockets für die Echtzeitübertragung von Simulationsdaten

Die Komponenten sind lose gekoppelt und kommunizieren über definierte Zustände und Ereignisse. Das Backend basiert auf FastAPI und umfasst zentrale Module wie `main.py` zur Bereitstellung der REST-Schnittstellen sowie `sumo_engine.py` zur Anbindung der externen Simulationsengine SUMO. Das Frontend beinhaltet zentrale GUI-Komponenten wie die Kartenansicht (`MapContainer.tsx`), die Werkzeugleiste (`AppToolbar.tsx`), die Fahrzeugkonfiguration (`VehiclePopup.tsx`) sowie die Fahrzeugdarstellung (`VehicleLayer.ts`).



## Visual Studio Code

### Implementierung des Backends

### Simulationssteuerung

### Kartenkomponente

### Werkzeugleiste

### Fahrzeugkonfiguration

### Fahrzeugverwaltung

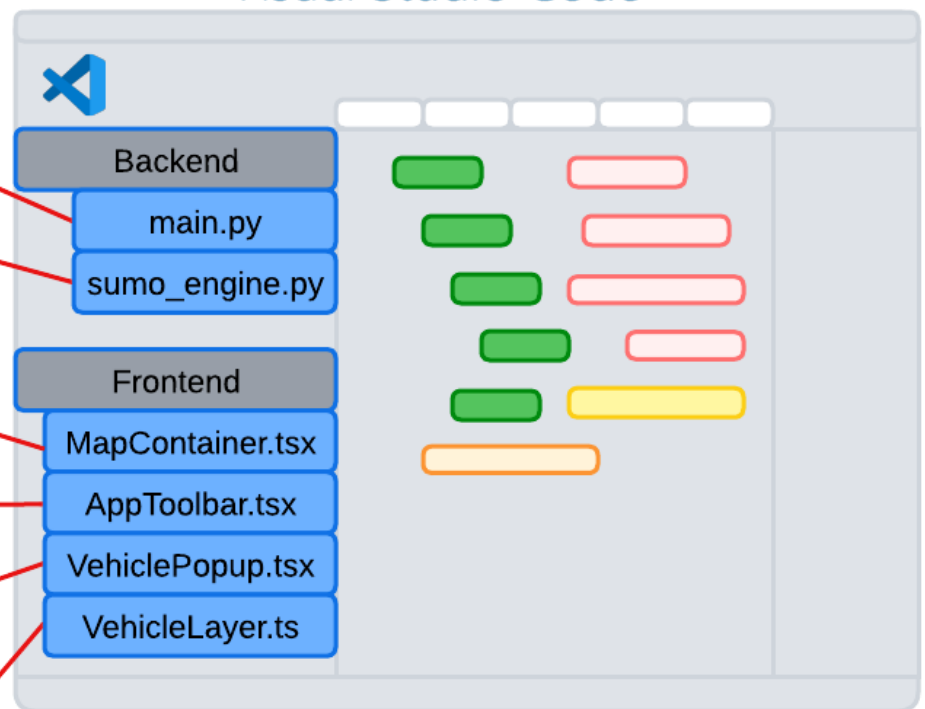


Abbildung 15: Die wichtigsten Module für die Verkehrssimulation

Die Abbildung zeigt die Implementierungsstruktur der entwickelten Verkehrssimulationsplattform mit einer klaren Trennung zwischen Frontend und Backend. Das Backend basiert auf FastAPI und umfasst zentrale Module wie `main.py` zur Bereitstellung der REST-Schnittstellen sowie `sumo_engine.py` zur Anbindung der externen Simulationsengine SUMO. Das Frontend wurde als komponentenbasierte Webanwendung mit React umgesetzt und beinhaltet zentrale GUI-Komponenten wie die Kartenansicht (`MapContainer.tsx`), die Werkzeugleiste (`AppToolbar.tsx`), die Fahrzeugkonfiguration (`VehiclePopup.tsx`) sowie die Fahrzeugdarstellung (`VehicleLayer.ts`). Die Abbildung verdeutlicht die modulare Struktur und unterstützt die Erweiterbarkeit sowie die Integration unterschiedlicher Simulationsengines. Die einzelnen Module und deren Funktionsweise werden in den folgenden Unterkapiteln detailliert beschrieben.

### 6.1.1 Kartenkomponente (MapTiler-Integration)

Die Kartenkomponente wurde mit MapTiler umgesetzt und bildet die zentrale Arbeitsfläche der Anwendung. Sie dient zur Darstellung des Straßennetzes, der Fahrzeuge und weiterer Verkehrselemente. Fahrzeugpositionen werden als Marker angezeigt und regelmäßig mit Daten aus dem Backend aktualisiert. Dadurch bleibt die Kartenansicht von der eigentlichen Simulationslogik getrennt.

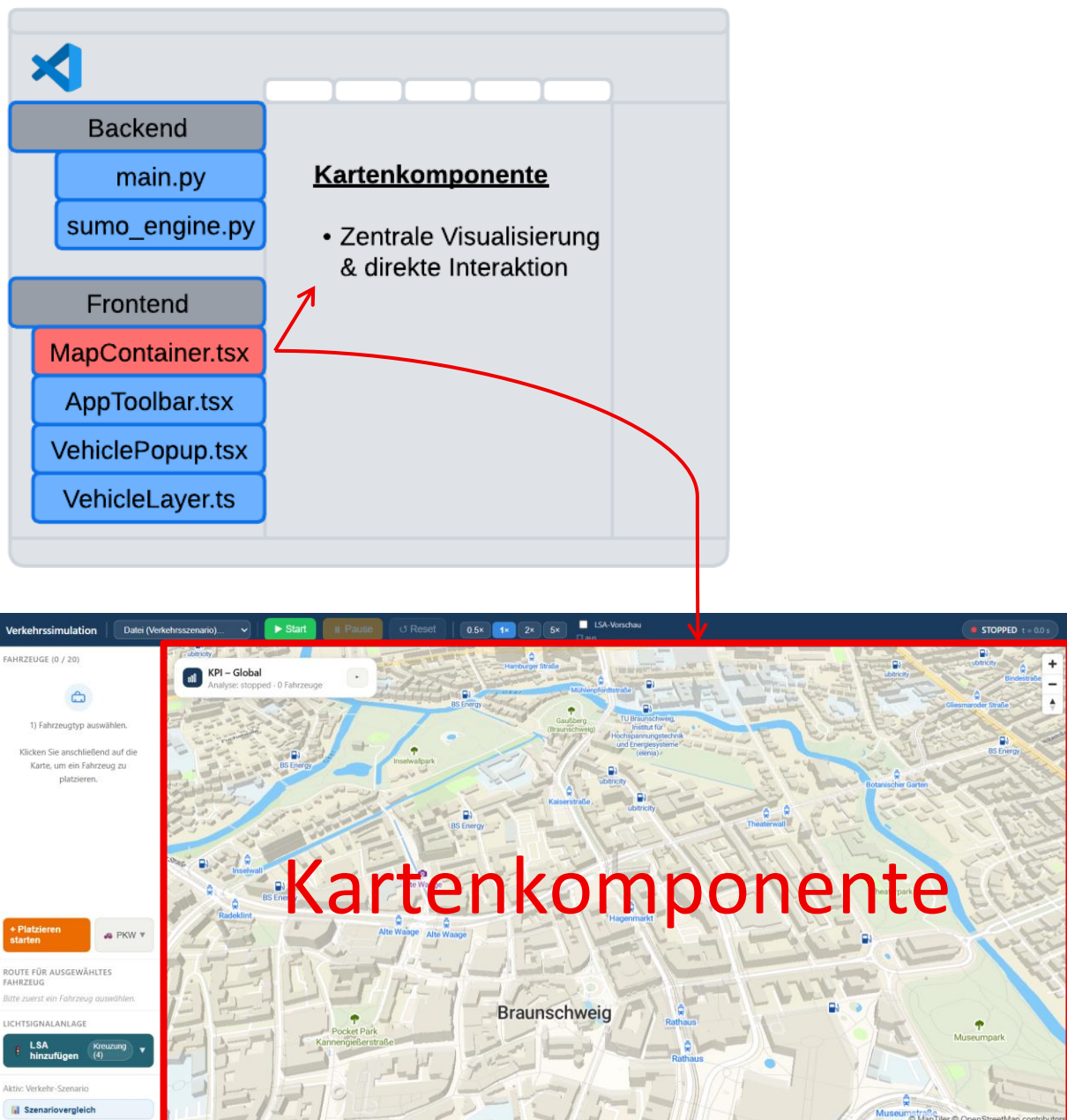


Abbildung 16: MapContainer.tsx (wichtigstes Modul der Kartenintegration)

Zusätzlich unterstützt die Karte Interaktionen wie das Setzen von Start- und Zielpunkten oder das Platzieren neuer Fahrzeuge. Entsprechende Klick- und Auswahlereignisse werden verarbeitet und an die zuständigen Komponenten weitergegeben.

### 6.1.2 Werkzeugleiste

Die Werkzeugleiste wurde als eigenständige und dauerhaft sichtbare Frontend-Komponente umgesetzt. Sie enthält zentrale Steuerelemente für den Simulationsablauf, darunter Start, Pause, Reset sowie die Anpassung der Simulationsgeschwindigkeit. Benutzeraktionen werden über REST-Schnittstellen an das Backend übermittelt und dort verarbeitet. Dadurch bleibt die Werkzeugleiste frei von Simulationslogik und kann unabhängig von der verwendeten Simulationsengine eingesetzt werden.



Abbildung 17: AppToolbar.tsx (zentrales Modul für Werkzeugleiste)



### 6.1.3 Fahrzeugverwaltung

Für die Verwaltung aktiver Fahrzeuge wurde eine eigene Übersichtskomponente implementiert. Dort werden alle aktuell vorhandenen Fahrzeuge mit grundlegenden Informationen wie Identifikationsnummer, Status und Fahrzeugtyp dargestellt. Die Komponente ermöglicht außerdem die Auswahl und Entfernung einzelner Fahrzeuge. Änderungen werden an das Backend übermittelt und mit dem globalen Simulationszustand synchronisiert.

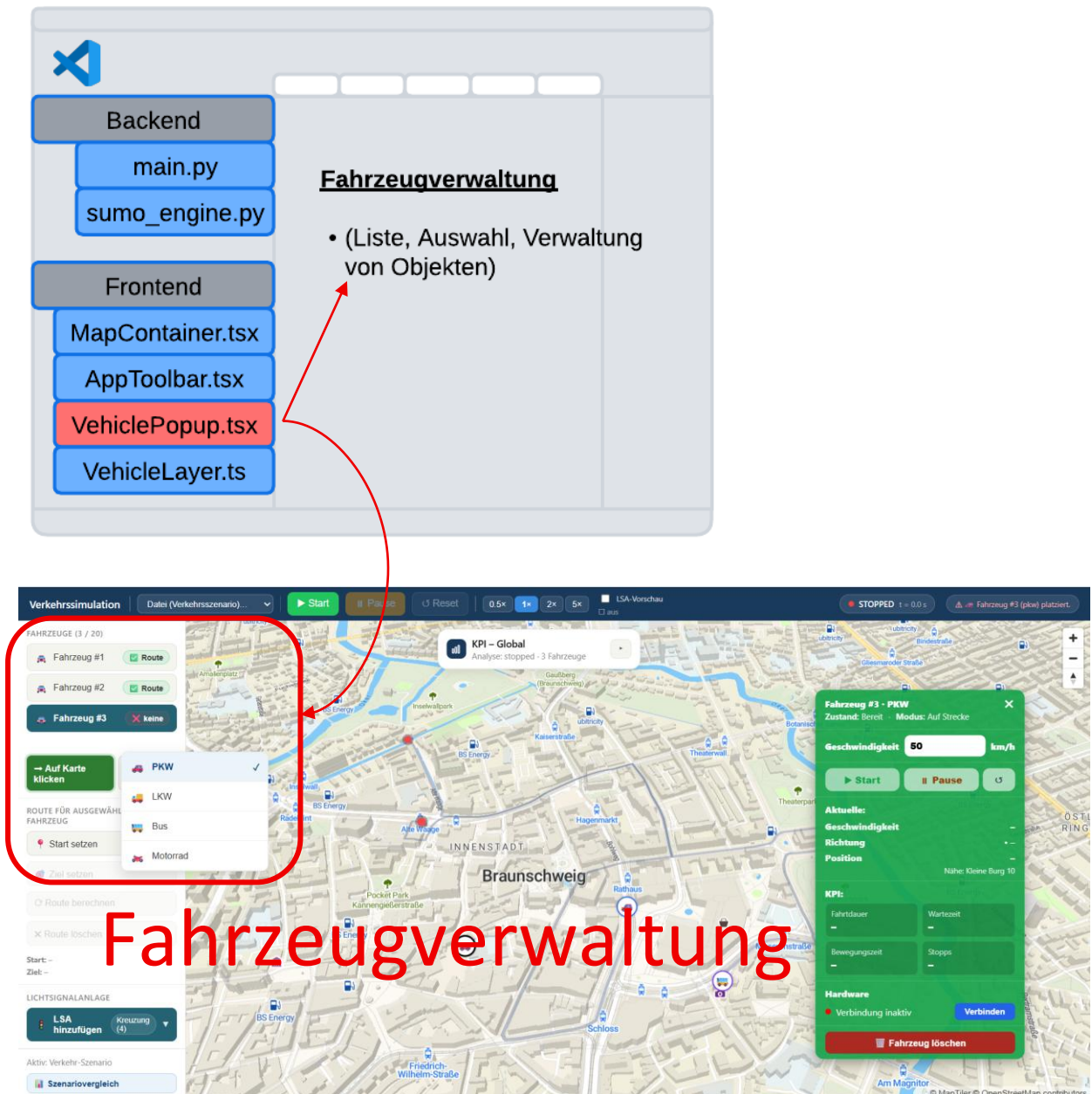


Abbildung 18: VehiclePopup.tsx (wichtigste Modul für Fahrzeugkonfiguration)

## 6.1.4 Fahrzeugkonfiguration

Die Konfiguration neuer Fahrzeuge wurde über ein kontextabhängiges Dialogsystem umgesetzt. Beim Hinzufügen eines Fahrzeugs werden grundlegende Parameter wie Fahrzeugtyp, Startposition und Zielpunkt erfasst. Start- und Zielpunkte können direkt über die Kartenansicht ausgewählt werden.

Nach Abschluss des Dialogs werden die Eingaben an das Backend übertragen und in den globalen Simulationszustand übernommen. Das Dialogsystem ist so aufgebaut, dass spätere Erweiterungen um zusätzliche Parameter möglich sind.

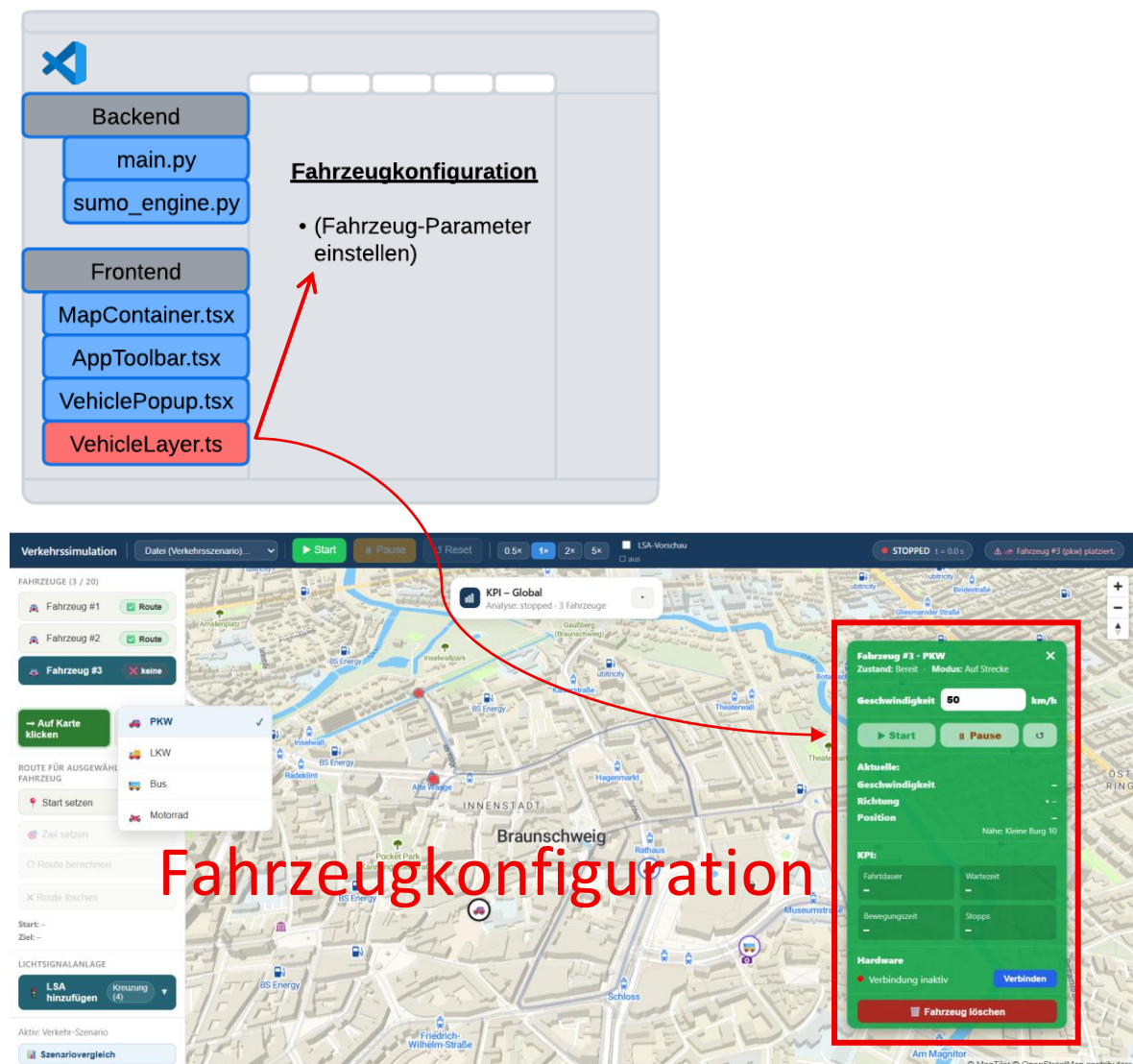


Abbildung 19: VehicleLayer.ts (wichtigste Modul für die Fahrzeugverwaltung)

## 6.2 Implementierung des Backends

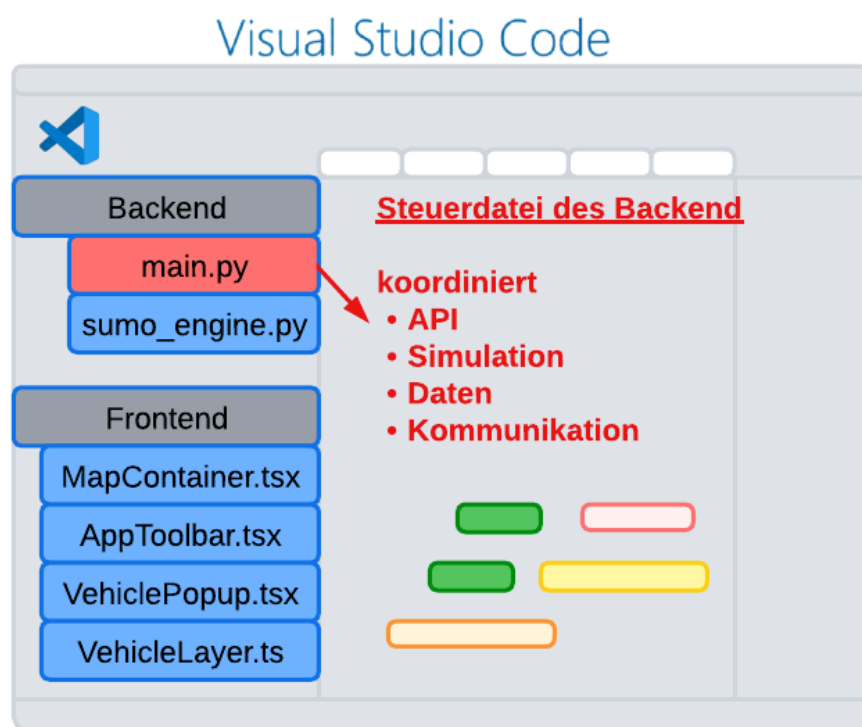
Das Backend übernimmt die zentrale Steuerung und Koordination der gesamten Verkehrssimulationsplattform. Es vermittelt zwischen Benutzeroberfläche, interner Simulation, externer Simulationsengine und optionaler Datenhaltung.

Die Implementierung erfolgte mit dem Python-Webframework FastAPI, das die Bereitstellung von REST-Schnittstellen sowie WebSocket-Kommunikation ermöglicht. Dadurch kann sowohl eine zustandsbasierte Steuerung als auch eine kontinuierliche Echtzeitübertragung von Simulationsdaten realisiert werden.

Das Backend erfüllt insbesondere folgende Aufgaben:

- Verwaltung des globalen Simulationszustands
- Verarbeitung von Steuerbefehlen aus dem Frontend
- Anbindung der internen Simulation
- Integration einer externen Simulationsengine (SUMO)
- Bereitstellung von Positions- und Statusdaten in Echtzeit

Die Architektur des Backends ist modular aufgebaut, um eine klare Trennung zwischen API-Ebene, Simulationslogik und Engine-Anbindung zu gewährleisten.



## 6.2.1 API-Struktur

Die folgende Abbildung zeigt die Kommunikationsstruktur zwischen Frontend, Backend und Simulationsengine. Sie verdeutlicht, wie Benutzeraktionen über REST-Schnittstellen an das Backend übertragen und anschließend an die Simulationslogik weitergeleitet werden.

REST-Endpunkte werden für zustandsverändernde Operationen verwendet. Dazu gehören beispielsweise:

- Erstellen oder Löschen von Fahrzeugen
- Starten, Pausieren und Zurücksetzen der Simulation
- Anpassung der Simulationsgeschwindigkeit
- Laden oder Speichern von Szenarien

Jede Anfrage wird vom Backend validiert und anschließend an die zuständige Simulationskomponente weitergeleitet. Die API ist so strukturiert, dass sie unabhängig von der konkret eingesetzten Simulationsengine funktioniert.

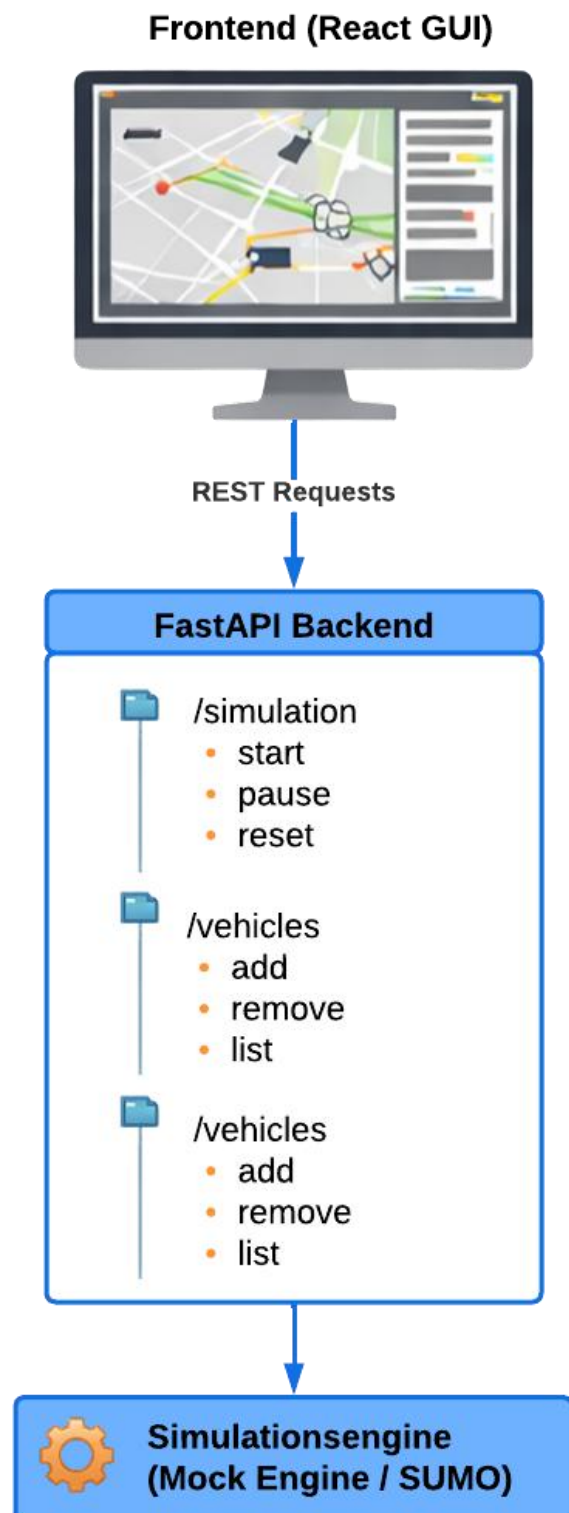


Abbildung 21: Struktur der REST-API



## 6.2.2 Simulationssteuerung

Die Simulationssteuerung wird zentral im Backend verwaltet. Hier wird der globale Simulationszustand gespeichert, der unter anderem folgende Informationen enthält:

- aktuelle Simulationszeit
- Simulationsstatus (laufend, pausiert, gestoppt)
- Simulationsgeschwindigkeit
- Liste aktiver Fahrzeuge

Steuerbefehle aus dem Frontend wie Start, Pause oder Reset verändern diesen Zustand. Je nach gewählter Simulationsquelle wird entweder der interne Simulationsloop ausgeführt oder eine externe Engine angesteuert. Bei der internen Simulation werden Fahrzeugpositionen in festen Zeitintervallen aktualisiert und anschließend per WebSocket an das Frontend übertragen. Bei einer externen Simulation empfängt das Backend die berechneten Zustände, verarbeitet sie und leitet sie an das Frontend weiter.

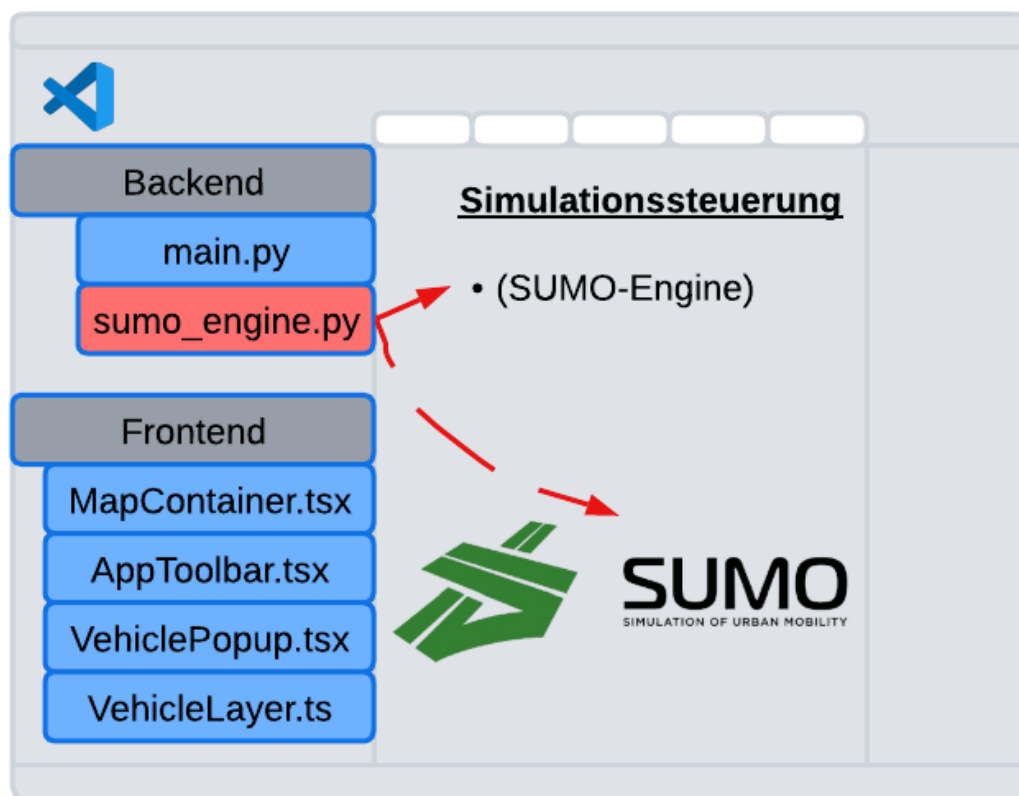


Abbildung 22: Module für die Simulationen

### 6.2.3 Abstraktion der Simulationsengine

Ein zentrales Ziel der Backend-Implementierung war die Entkopplung der Benutzeroberfläche von der konkreten Simulationsengine.

Hierzu wurde eine abstrakte Schnittstelle für Simulationsengines definiert. Diese Schnittstelle legt fest, welche Funktionen eine Engine bereitstellen muss, beispielsweise:

- Initialisierung der Simulation
- Starten und Stoppen
- Hinzufügen und Entfernen von Fahrzeugen
- Abrufen aktueller Fahrzeugzustände
- Fortschreiben der Simulation

Sowohl die interne Simulation als auch die externe SUMO-Anbindung implementieren diese Schnittstelle. Dadurch kann das Backend zur Laufzeit zwischen verschiedenen Engines wechseln, ohne dass Änderungen im Frontend erforderlich sind.

Diese Abstraktion bietet mehrere Vorteile:

- Austauschbarkeit der Simulationslogik
- klare Trennung von Verantwortlichkeiten
- Erweiterbarkeit um weitere Simulationsquellen
- Vorbereitung für die perspektivische Anbindung realer Komponenten

Die Abstraktion der Simulationsengine stellt somit einen zentralen Baustein der hybriden Systemarchitektur dar und ermöglicht eine flexible sowie langfristig erweiterbare Weiterentwicklung der Plattform.

### 6.3 Implementierung der internen Simulation

Die interne Simulation eignet sich besonders für Entwicklungs- und Testzwecke und dient als vereinfachte Referenzimplementierung innerhalb der hybriden Architektur. Sie ermöglicht es, Interaktionsabläufe, Echtzeitdarstellung und Systemintegration unabhängig von einer externen Simulationsengine zu testen. Im Gegensatz zu komplexen mikroskopischen Verkehrssimulationen liegt der Fokus hier nicht auf einer möglichst realistischen Modellierung des Verkehrsverhaltens, sondern auf einer stabilen, nachvollziehbaren und leicht erweiterbaren Simulationslogik.

Die interne Simulation übernimmt insbesondere folgende Aufgaben:

- Verwaltung aller aktiven Fahrzeuge
- Berechnung von Positionen und Geschwindigkeiten
- Fortschreibung der Simulation in diskreten Zeitschritten
- Bereitstellung aktueller Zustände für das Backend

Die Implementierung ist so gestaltet, dass sie die definierte Simulationsschnittstelle erfüllt und somit austauschbar mit einer externen Engine bleibt.

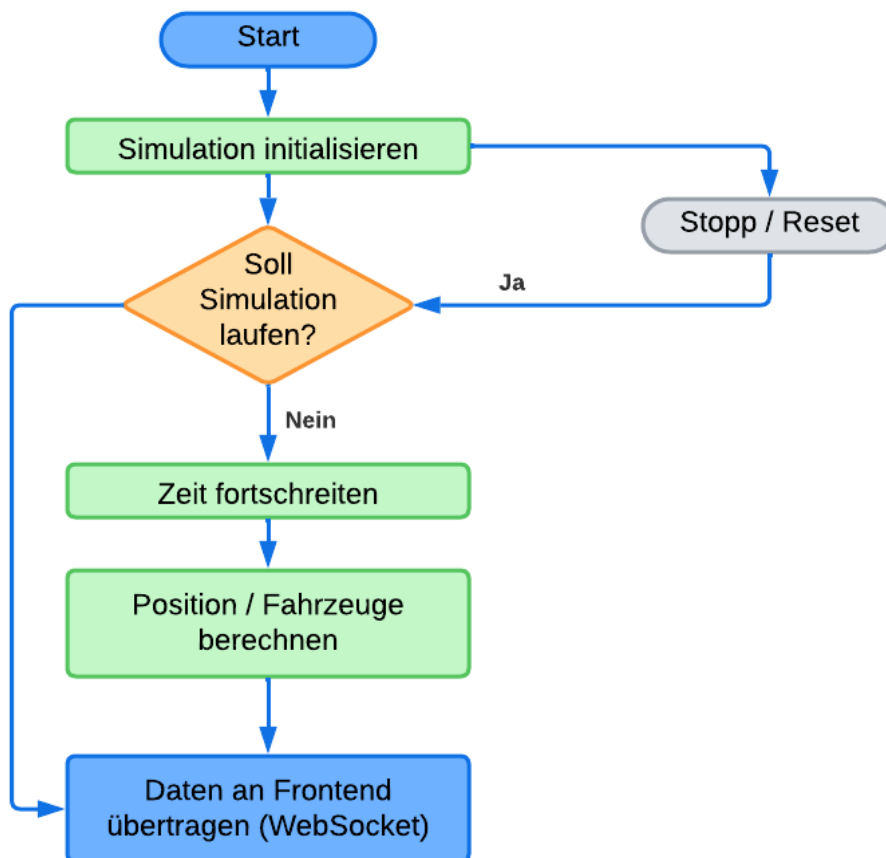


Abbildung 23: Ablaufdiagramm der internen Simulation

## 6.4 Integration einer externen Simulationsengine mit SUMO

Neben der in Kapitel 6.3 beschriebenen internen Simulation wurde in der Plattform zusätzlich die Verkehrssimulationssoftware SUMO (Simulation of Urban MObility) als externe Simulationsengine integriert. Ziel dieser Integration ist es, eine leistungsfähige mikroskopische Verkehrssimulation mit der entwickelten webbasierten Benutzeroberfläche zu verbinden und damit die hybride Systemarchitektur praktisch umzusetzen.

Während die interne Simulation vor allem für prototypische Tests, Demonstrationen und die Entwicklung der Benutzeroberfläche geeignet ist, ermöglicht SUMO eine deutlich realistischere Modellierung komplexer Verkehrsprozesse. Durch detaillierte Fahrzeugmodelle, umfangreiche Netzstrukturen und etablierte Simulationsmethoden wird SUMO in Forschung und Praxis häufig für wissenschaftliche Verkehrsuntersuchungen eingesetzt.

Die Integration von SUMO erfolgt innerhalb der entwickelten Architektur so, dass die Benutzeroberfläche weiterhin unabhängig von der konkreten Simulationsengine bleibt. Das Frontend kommuniziert ausschließlich mit dem Backend, welches wiederum als Vermittlungsschicht zwischen Benutzeroberfläche und Simulationsengine fungiert. Dadurch kann die Plattform sowohl mit der internen Simulation als auch mit externen Simulationssystemen betrieben werden, ohne dass Änderungen an der Benutzeroberfläche erforderlich sind.

26/27

---

<sup>26</sup> Simulation of Urban MObility. (kein Datum). SUMO User Documentation.

<sup>27</sup> Docker. (kein Datum). Install Docker Desktop on Windows.

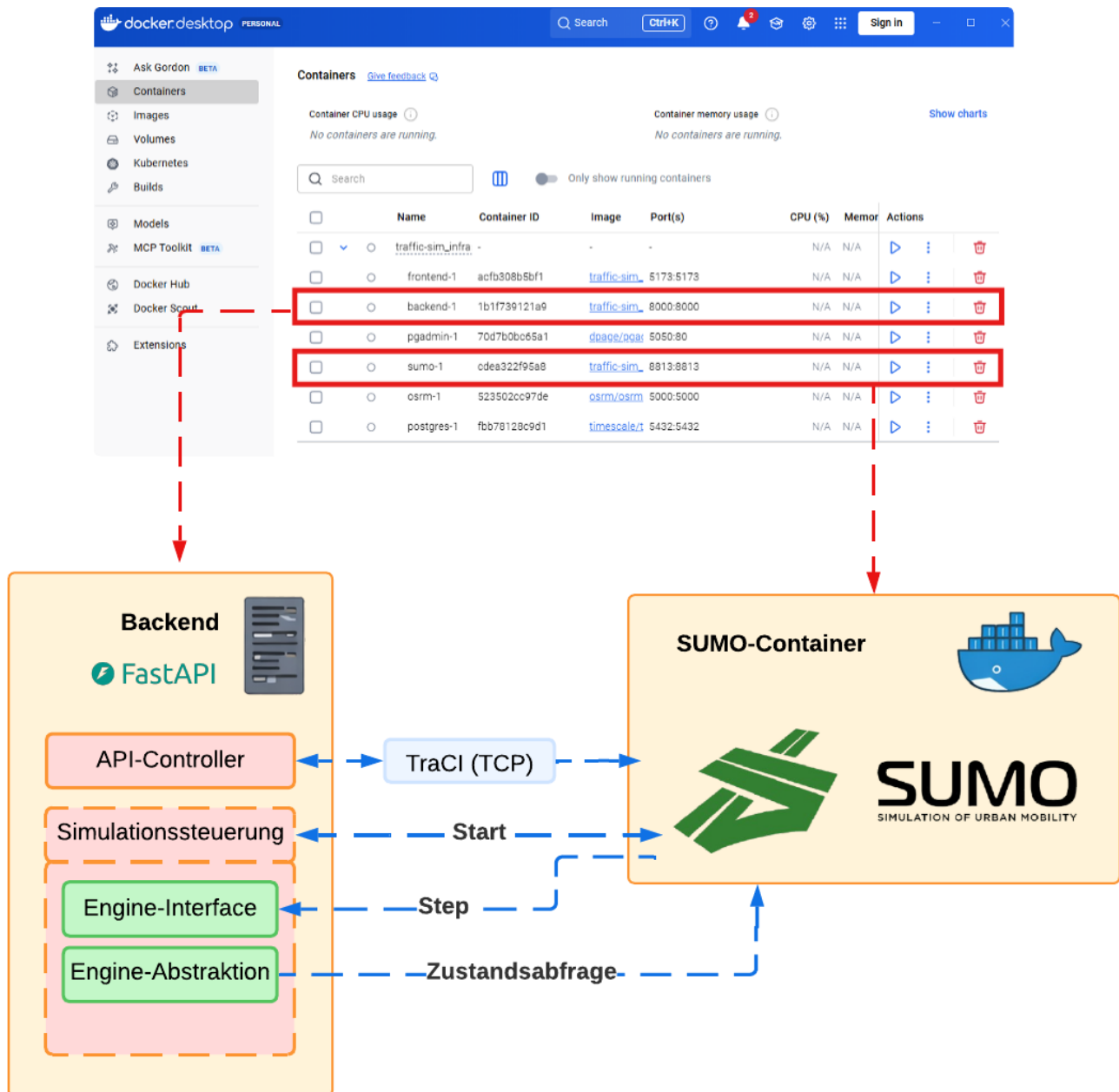


Abbildung 24: SUMO-Integration in einer Docker Umgebung

Abbildung „SUMO-Integration in einer Docker Umgebung“ zeigt die Architektur der Integration von SUMO in die entwickelte Simulationsplattform. Das Backend, implementiert mit FastAPI, übernimmt die Steuerung der Simulation und kommuniziert über das Traffic Control Interface (TraCI) mit der SUMO-Simulation, die in einem separaten Docker-Container ausgeführt wird. Über diese Schnittstelle können Simulationsschritte ausgelöst sowie Fahrzeugzustände abgefragt werden. Durch die Verwendung einer abstrakten Schnittstelle für Simulationsengines bleibt die Systemarchitektur unabhängig von der konkret eingesetzten Simulationsengine.

### 6.4.1 Kommunikation über TraCI

Die Kommunikation zwischen dem Backend der entwickelten Plattform und SUMO erfolgt über das Traffic Control Interface (TraCI). TraCI ist eine TCP-basierte Schnittstelle, die es ermöglicht, eine laufende SUMO-Simulation von außen zu steuern und gleichzeitig Zustandsinformationen auszulesen. Über TraCI können verschiedene Steuer- und Abfrageoperationen ausgeführt werden, beispielsweise das Hinzufügen oder Entfernen von Fahrzeugen, das Abfragen von Fahrzeugpositionen oder das Verändern von Zuständen der Verkehrsinfrastruktur.

28

Die Kommunikation zwischen Backend und SUMO erfolgt in fünf Schritten:

1. Backend sendet TraCI-Befehl zur Ausführung eines Simulationsschritts.
2. SUMO berechnet nächsten Zeitschritt der Simulation.
3. Backend fragt aktuelle Zustandsdaten ab (Fahrzeugpositionen, Geschwindigkeit, Status).
4. Die Zustandsdaten werden von SUMO an das Backend zurückgegeben.
5. Backend überträgt die Zustände an das Frontend

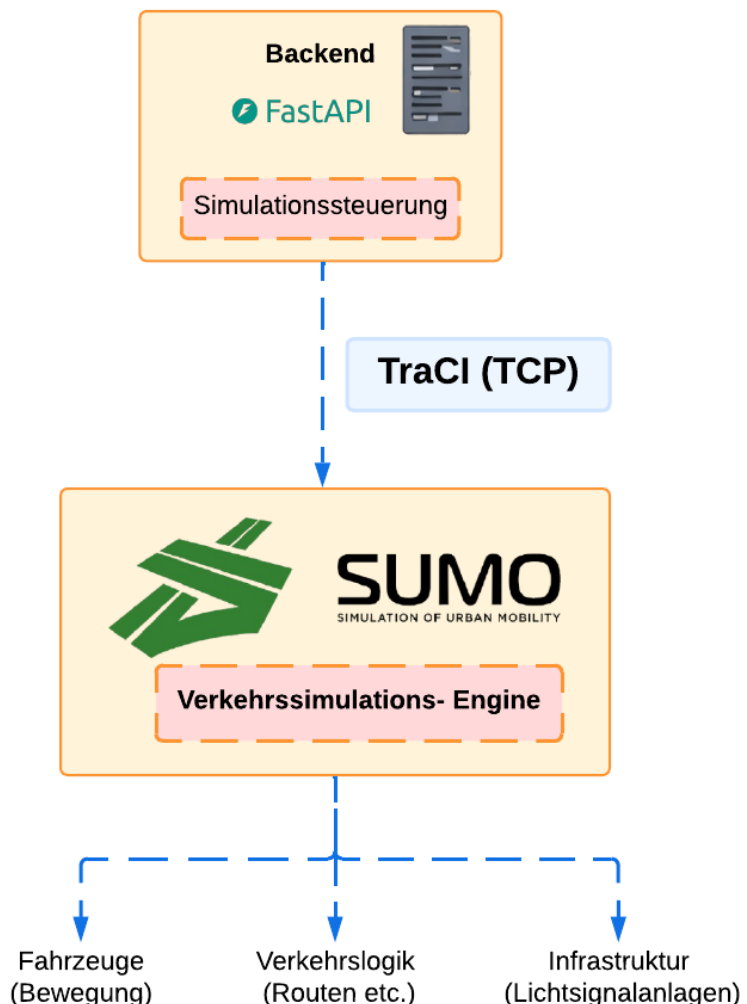


Abbildung 25: Kommunikationsablauf zwischen Backend und SUMO über TraCI

<sup>28</sup> Mo, T. (kein Datum). Web-based Visualization and Interaction of SUMO Traffic Simulation in Digital Twin Model.

### 6.4.2 Implementierung der SUMO-Anbindung

Die SUMO-Integration wurde so implementiert, dass sie die zuvor definierte Engine-Schnittstelle erfüllt. Zur Gewährleistung einer reproduzierbaren Umgebung wird SUMO containerbasiert betrieben: SUMO läuft in einem separaten Docker-Container, während das Backend in einem eigenen Container ausgeführt wird. Die Kommunikation erfolgt über ein internes Docker-Netzwerk. Beim Start einer Simulation wird SUMO mit definierten Konfigurationsdateien gestartet, eine TraCI-Verbindung aufgebaut und die Simulation initialisiert. Das Backend übernimmt anschließend die Steuerung der Simulationsschritte und koordiniert Fahrzeugerstellung, Zustandsabfragen und die Synchronisation mit dem Frontend. Durch die Integration in die bestehende Backend-Architektur bleibt die REST- und WebSocket-Schnittstelle für das Frontend unverändert.

Nach jedem Simulationsschritt werden relevante Fahrzeugdaten aus SUMO abgefragt – darunter aktuelle Position, Geschwindigkeit, Fahrspur und Status. Diese Daten werden im Backend in den globalen Simulationszustand übernommen, über WebSocket an das Frontend übertragen und in der Kartenansicht visualisiert. Dadurch entsteht eine kontinuierliche Echtzeitdarstellung der SUMO-Simulation innerhalb der webbasierten Benutzeroberfläche.

## 6.5 Vergleich der Simulationsvarianten und

### Erweiterungspotenziale

Beide in der Plattform unterstützten Simulationsvarianten nehmen innerhalb der hybriden Architektur unterschiedliche, sich ergänzende Rollen ein. Die interne Simulation ist bewusst einfach gehalten und eignet sich vor allem für prototypische Tests, Demonstrationen und die Entwicklung der Benutzeroberfläche. Aufgrund ihrer vereinfachten Modellierung ist sie jedoch nur eingeschränkt für realitätsnahe Verkehrsuntersuchungen geeignet. SUMO bietet dagegen eine höhere Realitätsnähe und eignet sich besser für wissenschaftlich fundierte Verkehrsuntersuchungen, erfordert jedoch einen höheren Konfigurationsaufwand.

Die entwickelte Architektur ist bewusst modular aufgebaut, sodass weitere Simulationsquellen oder Systemkomponenten integriert werden können. Mögliche Erweiterungen der Plattform umfassen:

- Integration zusätzlicher externer Simulationsengines
- Kopplung mit realen Modellfahrzeugen oder mobilen Robotern
- Integration von KI-basierten Steuerungsmodulen
- Co-Simulation mehrerer Simulationssysteme
- Verteilte Simulation über mehrere Instanzen

Langfristig kann die Plattform somit als flexible Integrationsumgebung für unterschiedliche Verkehrsmodelle sowie experimentelle Laboraufbauten dienen.

| Kriterium             | <div>  <div>Interne Simulation (Mock-Engine)</div> </div> <div>vs</div> <div> <div>SUMO Simulation</div>  </div> |                           |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
|                       |                                                                                                                                                                                                                                                                                          |                           |
| Komplexität           | Gering                                                                                                                                                                                                                                                                                   | hoch                      |
| Realitätsnähe         | vereinfacht                                                                                                                                                                                                                                                                              | mikroskopisch detailliert |
| Performance           | sehr hoch                                                                                                                                                                                                                                                                                | mikroskopisch detailliert |
| Konfigurationsaufwand | gering                                                                                                                                                                                                                                                                                   | höher                     |
| Forschungsrelevanz    | begrenzt                                                                                                                                                                                                                                                                                 | hoch                      |

Abbildung 26: Vergleich zwischen interner Simulation und SUMO



## 7. Evaluation

Ziel der Evaluation ist es, die entwickelte Verkehrssimulationsplattform systematisch im Hinblick auf die in Kapitel 4 definierten funktionalen und nichtfunktionalen Anforderungen zu bewerten. Dabei wird insbesondere untersucht, inwieweit die Plattform eine interaktive, benutzerfreundliche und erweiterbare Nutzung von Verkehrssimulationen ermöglicht und ob sie für nicht-spezialisierte Nutzer in Lehr- und Laborumgebungen geeignet ist.

Die Evaluation gliedert sich in drei Teile:

1. Funktionale Tests zentraler Systemfunktionen
2. Explorativer Usability-Test mit Kommilitonen
3. Qualitativer Vergleich mit der bestehenden SUMO-GUI und Diskussion der Ergebnisse

### 7.1 Funktionale Tests

Zur Überprüfung der funktionalen Anforderungen wurden zentrale Anwendungsfälle der Plattform systematisch getestet. Ziel war es, die korrekte Umsetzung der Kernfunktionen sowie das Zusammenspiel der einzelnen Systemkomponenten zu validieren.

Die Ergebnisse zeigen, dass alle zentralen funktionalen Anforderungen erfüllt wurden. Benutzeraktionen werden korrekt vom Frontend an das Backend übertragen und führen zu konsistenten Zustandsänderungen. Die kontinuierliche Übertragung von Simulationsdaten über WebSockets ermöglicht eine stabile Echtzeitdarstellung. Sowohl die interne Simulation als auch die SUMO-Integration konnten erfolgreich genutzt werden, ohne Anpassungen an der Benutzeroberfläche vorzunehmen.

Die folgende Tabelle zeigt die getesteten Anforderungen aus Kapitel 4 und deren Ergebnis:

*Tabelle 4: Funktionale Testergebnisse*

| Nr. | Anforderung                      | Testbeschreibung                                     | Ergebnis  |
|-----|----------------------------------|------------------------------------------------------|-----------|
| 1   | Straßennetz darstellen           | Kartenansicht laden, Zoom und Pan prüfen             | ✓ Erfüllt |
| 2   | Fahrzeuge platzieren             | Fahrzeug per Klick in Karte platzieren               | ✓ Erfüllt |
| 3   | Fahrzeugtypen auswählen          | Auswahl zwischen PKW und LKW prüfen                  | ✓ Erfüllt |
| 4   | Mehrere Fahrzeuge verwalten      | Mehrere Fahrzeuge hinzufügen und in Liste anzeigen   | ✓ Erfüllt |
| 5   | Simulation starten               | Start-Button betätigen, Positionsupdates beobachten  | ✓ Erfüllt |
| 6   | Simulation pausieren             | Pause-Button betätigen, Zustand prüfen               | ✓ Erfüllt |
| 7   | Simulation zurücksetzen          | Reset-Button betätigen, Ausgangszustand prüfen       | ✓ Erfüllt |
| 8   | Geschwindigkeit anpassen         | Geschwindigkeitsregler betätigen, Auswirkung prüfen  | ✓ Erfüllt |
| 9   | Interne + externe Engine nutzbar | Wechsel zwischen interner Simulation und SUMO testen | ✓ Erfüllt |

Im Rahmen der Tests wurden jedoch auch Grenzen der Plattform sichtbar. Bei der SUMO-Integration traten gelegentlich Verzögerungen beim Verbindungsaufbau über TraCI auf, wenn SUMO noch nicht vollständig initialisiert war. Zudem zeigte die WebSocket-Verbindung bei einer größeren Anzahl gleichzeitig aktiver Fahrzeuge eine leicht erhöhte Latenz bei der Positionsübertragung. Diese Beobachtungen stellen keine kritischen Fehler dar, zeigen aber Optimierungspotenzial für zukünftige Weiterentwicklungen auf.

## 7.2 Explorativer Usability-Test

Ergänzend zu den funktionalen Tests wurde ein explorativer Usability-Test mit bis zu fünfzehn Kommilitonen durchgeführt. Der Test folgt dem Ansatz nach Nielsen (1993), der zeigt, dass bereits fünf Testpersonen rund 85 % der relevanten Usability-Probleme aufdecken können, weshalb die gewählte Stichprobengröße für eine explorative Untersuchung methodisch angemessen ist. Ziel war es, erste Beobachtungen zur Bedienbarkeit der Plattform aus Nutzerperspektive zu sammeln. Der Test wurde nach einem einheitlichen Ablaufschema durchgeführt: Jede Testperson erhielt dieselben drei Aufgaben in schriftlicher Form ohne weitere mündliche Erläuterung. Beobachtungen zu Bedienfehlern, Suchverhalten und Reaktionen wurden während des Tests stichpunktartig notiert. Als informelles Abbruchkriterium galt, wenn eine Testperson eine Aufgabe nach mehr als fünf Minuten nicht abschließen konnte – dieser Fall trat bei keiner der Testpersonen ein. Die Testpersonen hatten keine Vorerfahrung mit der entwickelten Plattform und wurden gebeten, folgende drei Aufgaben ohne Anleitung durchzuführen:

1. Ein Fahrzeug auf der Karte platzieren
2. Die Simulation starten und pausieren
3. Die Simulationsgeschwindigkeit anpassen

Die Testpersonen benötigten durchschnittlich ein bis zwei Minuten, um alle drei Aufgaben erfolgreich abzuschließen. Dabei wurden keine wesentlichen Bedienprobleme beobachtet. Die Steuerelemente wie Start, Pause und Reset wurden von allen Testpersonen ohne Erklärung gefunden und korrekt bedient. Das Platzieren von Fahrzeugen über die Kartenansicht wurde als intuitiv empfunden.

Einschränkend ist anzumerken, dass die Testpersonen als Studierende eines technischen Studiengangs zwar grundlegende digitale Kompetenz mitbringen, jedoch nicht vollständig repräsentativ für alle Zielgruppen der Plattform sind. Insbesondere Lehrende ohne informatischen Hintergrund oder Labormitarbeitende mit spezifischen Anforderungen an die Hardwareanbindung wurden nicht einbezogen. Dies stellt eine methodische Einschränkung der vorliegenden Studie dar. Eine umfassendere Nutzerstudie mit heterogeneren Testgruppen und standardisiertem Testprotokoll wäre für eine belastbare Aussage über die Gebrauchstauglichkeit der Plattform in realen Lehr- und Laborumgebungen erforderlich und stellt einen Ansatz für zukünftige Arbeiten dar.

### Quantitative Usability-Bewertung (UMUX-Lite)

Ergänzend zu den qualitativen Beobachtungen wurde ein erweiterter UMUX-Lite-Fragebogen mit denselben Testpersonen durchgeführt. UMUX-Lite ist ein wissenschaftlich validiertes Kurzinstrument zur Usability-Messung nach Finstad (2010). Der klassische Fragebogen umfasst zwei Kernaussagen zu wahrgenommenem Funktionsumfang und Bedienbarkeit. In dieser Studie wurde ergänzend eine dritte Frage zur Weiterempfehlungsbereitschaft hinzugefügt, um die subjektive Gesamtbewertung der Plattform vollständiger abzubilden. Die Testpersonen bewerteten folgende Aussagen auf einer Skala von 1 (stimme überhaupt nicht zu) bis 5 (stimme völlig zu):

- „Die Funktionen der Verkehrssimulationsplattform erfüllen meine Anforderungen.“
- „Die Verkehrssimulationsplattform ist einfach zu bedienen.“
- „Ich würde die Verkehrssimulationsplattform weiterempfehlen.“

*Tabelle 5: UMUX-Lite Ergebnisse*

| Person | Frage 1 | Frage 2 | Frage 3 | Summe |
|--------|---------|---------|---------|-------|
| 1      | 4       | 5       | 4       | 13    |
| 2      | 5       | 4       | 5       | 14    |
| 3      | 4       | 4       | 4       | 12    |
| 4      | 5       | 5       | 5       | 15    |
| 5      | 5       | 5       | 5       | 15    |
| 6      | 4       | 5       | 4       | 13    |
| 7      | 5       | 4       | 4       | 13    |
| 8      | 4       | 4       | 5       | 13    |
| 9      | 5       | 5       | 4       | 14    |
| 10     | 4       | 4       | 4       | 12    |
| 11     | 5       | 5       | 5       | 15    |
| 12     | 4       | 4       | 5       | 13    |

|          |            |            |            |             |
|----------|------------|------------|------------|-------------|
| 13       | 5          | 4          | 4          | 13          |
| 14       | 4          | 5          | 5          | 14          |
| 15       | 5          | 4          | 4          | 13          |
| <b>Ø</b> | <b>4,5</b> | <b>4,5</b> | <b>4,5</b> | <b>13,5</b> |

29

Der erzielte Gesamtwert von durchschnittlich 13,5 von maximal 15 Punkten (Ø pro Frage: 4,5 von 5) entspricht nach der von Finstad (2010) beschriebenen Umrechnungsformel einem SUS-äquivalenten Score von etwa 85–90, was nach gängiger Bewertungsskala als „gut bis sehr gut“ eingestuft wird. Die ergänzte dritte Frage zur Weiterempfehlungsbereitschaft (Ø 4,5) bestätigt die positive Grundwahrnehmung der Plattform. Mit 15 Testpersonen ist die Stichprobengröße gegenüber der ursprünglichen Planung erweitert worden, bleibt jedoch aufgrund der eingeschränkten Repräsentativität der Testgruppe explorativ. Die Ergebnisse sind daher nicht statistisch generalisierbar und sollten als erste orientierende Einschätzung verstanden werden.

---

<sup>29</sup> Finstad, K. (06. Mai 2010). The Usability Metric for User Experience.

## 7.3 Vergleich GUI vs. SUMO-GUI

Zur Einordnung der entwickelten Benutzeroberfläche wurde ein qualitativer Vergleich mit der bestehenden SUMO-GUI durchgeführt. Die Vergleichskriterien wurden auf Basis der in Kapitel 4 definierten Anforderungen sowie der Usability-Heuristiken nach Nielsen (1993) ausgewählt. Die Kriterien Erlernbarkeit und Effizienz sind dabei direkt den Heuristiken von Nielsen entnommen; das Kriterium Einstiegshürde leitet sich aus der Anforderung einer niedrigschwelligen Nutzung für nicht-spezialisierte Anwender ab (vgl. Kap. 1.2). Die Bewertung der entwickelten GUI stützt sich zusätzlich auf die Beobachtungen des Usability-Tests sowie die UMUX-Lite-Ergebnisse aus Kapitel 7.2, die eine extern erhobene Nutzerperspektive einbringen. Der Vergleich basiert auf folgenden Kriterien:

*Tabelle 6: Vergleich GUI vs. SUMO-GUI*

| Kriterium       | Entwickelte GUI                                  | SUMO-GUI                                  |
|-----------------|--------------------------------------------------|-------------------------------------------|
| Einstiegshürde  | Gering – keine Konfigurationsdateien notwendig   | Hoch – technisches Vorwissen erforderlich |
| Interaktivität  | Änderungen direkt während der Simulation möglich | Änderungen meist nur vor Simulationsstart |
| Installation    | Keine – läuft direkt im Browser                  | Lokale Installation erforderlich          |
| Transparenz     | Simulationsstatus jederzeit sichtbar             | Detailliert, aber komplex                 |
| Funktionsumfang | Bewusst reduziert auf Kernfunktionen             | Umfangreich, für Experten geeignet        |

Die entwickelte GUI weist eine deutlich geringere Einstiegshürde auf, da keine vorbereitenden Konfigurationsschritte erforderlich sind. Änderungen können direkt während der laufenden Simulation vorgenommen werden, was experimentelles Arbeiten erleichtert. Diese Einschätzung wird durch die Ergebnisse des Usability-Tests gestützt: Die 15 Testpersonen bewerteten die Plattform im erweiterten UMUX-Lite-Fragebogen mit einem Durchschnitt von 4,5 von 5 Punkten pro Frage, was auf eine hohe wahrgenommene Bedienbarkeit und Weiterempfehlungsbereitschaft hindeutet. Die SUMO-GUI bietet

dagegen einen deutlich größeren Funktionsumfang und ist für detaillierte wissenschaftliche Analysen besser geeignet – entsprechend der Heuristik der Effizienz für erfahrene Nutzer nach Nielsen (1993). Die entwickelte Plattform ersetzt SUMO dabei nicht, sondern ergänzt es durch eine zugängliche Benutzeroberfläche für Lehr- und Laborumgebungen.

## 7.4 Diskussion der Ergebnisse

Die Ergebnisse der Evaluation zeigen, dass die zentrale Forschungsfrage dieser Arbeit beantwortet werden kann: Eine einfache webbasierte Benutzeroberfläche für SUMO kann erfolgreich entwickelt und gestaltet werden, die nicht-spezialisierte Nutzer in Lehr- und Laborumgebungen unterstützt.

Die funktionalen Tests bestätigen, dass alle definierten Kernanforderungen umgesetzt wurden. Der explorative Usability-Test mit 15 Kommilitonen zeigt, dass nicht-spezialisierte Nutzer die Plattform innerhalb von ein bis zwei Minuten ohne Anleitung bedienen können. Der erweiterte UMUX-Lite-Fragebogen mit drei Fragen ergibt einen Durchschnittswert von 4,5 von 5 Punkten pro Frage, was einem SUS-äquivalenten Score von etwa 85–90 entspricht und auf eine sehr positive Nutzerwahrnehmung hinweist. Der Vergleich mit der SUMO-GUI verdeutlicht, dass die entwickelte Lösung insbesondere in Bezug auf Zugänglichkeit und Interaktivität Vorteile bietet.

Einordnung in den Forschungskontext: Mo (2026) entwickelte im Rahmen einer verwandten Arbeit ebenfalls eine webbasierte Visualisierungsschicht für SUMO-Simulationen, fokussierte dabei jedoch primär auf die reine Darstellung von Simulationsdaten in einem Digital-Twin-Kontext. Die vorliegende Arbeit geht darüber hinaus, indem sie zusätzlich eine interaktive Steuerungsebene sowie eine abstrakte Engine-Schnittstelle realisiert, die den Wechsel zwischen unterschiedlichen Simulationsquellen ohne Anpassungen an der Benutzeroberfläche ermöglicht. Dies stellt einen eigenständigen architektonischen Beitrag dar, der in vergleichbaren Arbeiten bisher nicht in dieser Form umgesetzt wurde. Gleichzeitig bestehen Einschränkungen. Die interne Simulation ist bewusst vereinfacht und eignet sich nicht für komplexe realitätsnahe Verkehrsanalysen – detaillierte wissenschaftliche Untersuchungen erfordern weiterhin den direkten Einsatz von SUMO. Die implementierten Fahrzeugtypen beschränken sich auf PKW und LKW; weitere Typen wie Busse oder Fahrräder wurden konzeptionell vorgesehen, aber noch nicht umgesetzt. Bei einer größeren Anzahl gleichzeitig aktiver Fahrzeuge wurde zudem eine leicht erhöhte Latenz bei der WebSocket-Übertragung beobachtet, was Optimierungspotenzial für zukünftige Versionen aufzeigt. Schließlich wurde keine reale Hardware in die Plattform eingebunden, sodass die Hybridfähigkeit bisher ausschließlich konzeptionell nachgewiesen ist. Eine umfassendere Nutzerstudie mit heterogeneren Testgruppen bleibt als Ansatz für zukünftige Arbeiten vorgesehen.



## 8. Fazit

Der wissenschaftliche Beitrag dieser Arbeit liegt insbesondere in der Konzeption und prototypischen Umsetzung einer modularen webbasierten Steuerungs- und Visualisierungsschicht für Verkehrssimulationen. Zentrales architektonisches Merkmal ist die abstrakte Engine-Schnittstelle, die einen transparenten Wechsel zwischen unterschiedlichen Simulationsquellen ermöglicht, ohne Änderungen an der Benutzeroberfläche vorzunehmen. Diese Entkopplung von Darstellung und Simulationslogik stellt einen eigenständigen Beitrag zur Gestaltung erweiterbarer Simulationsplattformen dar. Im Sinne des Design Science Research nach Hevner et al. (2004) wurde ein Artefakt systematisch identifiziert, entwickelt und evaluiert: Die funktionalen Tests belegen die korrekte Umsetzung aller definierten Anforderungen, und der Usability-Test mit 15 Testpersonen weist die praktische Nutzbarkeit des Systems nach. Damit ist die Nützlichkeit des Artefakts im Sinne des DSR-Rahmenwerks belegt.

Zusammenfassend zeigt diese Arbeit, dass eine webbasierte Benutzeroberfläche den Zugang zu Verkehrssimulationen deutlich vereinfachen kann. Durch die Kombination moderner Webtechnologien mit einer modularen Systemarchitektur konnte eine Plattform entwickelt werden, die eine interaktive Steuerung und Visualisierung von Verkehrsszenarien ermöglicht und sowohl eine interne Simulation als auch eine externe Simulationsengine wie SUMO integriert. Die entwickelte Architektur trennt klar zwischen Benutzeroberfläche, Backend und Simulationslogik. Dadurch bleibt die Benutzeroberfläche unabhängig von der verwendeten Simulationsengine und kann flexibel mit unterschiedlichen Simulationsquellen betrieben werden. Die Evaluation mit 15 Testpersonen und einem erweiterten UMUX-Lite-Fragebogen zeigt, dass die zentralen funktionalen Anforderungen umgesetzt wurden und die Plattform eine intuitive Erstellung, Steuerung und Beobachtung von Verkehrsszenarien in Echtzeit ermöglicht. Der erzielte Durchschnittswert von 4,5 von 5 Punkten je Frage bestätigt eine hohe Nutzerzufriedenheit.

Der Vergleich mit der bestehenden SUMO-GUI verdeutlicht, dass die entwickelte Lösung nicht die vollständige Funktionsvielfalt etablierter Simulationswerkzeuge ersetzen soll. Vielmehr ergänzt sie diese durch eine leicht zugängliche und interaktive Benutzeroberfläche, die insbesondere für Lehrveranstaltungen, experimentelle Untersuchungen und prototypische Anwendungen geeignet ist. Die Plattform kann direkt im Webbrowser ohne Installation genutzt werden und eignet sich damit unmittelbar für den Einsatz in

Lehrveranstaltungen, beispielsweise an der Ostfalia Hochschule, sowie in Indoor-Laborumgebungen mit Modellfahrzeugen. Für zukünftige Arbeiten ergeben sich mehrere priorisierbare Erweiterungsmöglichkeiten. Als unmittelbarer nächster Schritt bietet sich die Anbindung eines ersten realen Modellfahrzeugs über das bestehende Hardware-Interface an, da die Architektur hierfür bereits konzeptionell vorbereitet ist. Mittelfristig können weitere Fahrzeugtypen, komplexere Routenplanungsalgorithmen sowie eine tiefere Einbindung von SUMO-Funktionalitäten wie Lichtsignalanlagen oder Spurwechselmodelle integriert werden. Langfristig ermöglicht die modulare Architektur eine Co-Simulation mehrerer Simulationssysteme sowie den Einsatz KI-basierter Steuerungsmodule, etwa zur autonomen Verkehrsoptimierung. Die entwickelte Architektur bietet für all diese Erweiterungen eine geeignete und stabile Grundlage.

Insgesamt zeigt die Arbeit, dass eine modulare, webbasierte Steuerungs- und Visualisierungsschicht die Nutzbarkeit bestehender Verkehrssimulationssysteme verbessern und neue Einsatzmöglichkeiten in Lehre, Forschung und Laborumgebungen eröffnen kann.

## Literaturverzeichnis

- aimsun. (kein Datum). *Aimsun delivers software and services to support transport planning and operations*. Abgerufen am 17. Dezember 2025 von <https://www.aimsun.com/>
- Baza-Solares, N.-M. R.-B. (kein Datum). *Traffic Simulation with Open-Source and Commercial Traffic Microsimulators: A Case Study*. Abgerufen am 3. Januar 2026 von <https://escholarship.org/content/qt38n4716p/qt38n4716p.pdf>
- Bundesanstalt für Straßen- und Verkehrswesen. (kein Datum). *Fahrsimulator*. Abgerufen am 4. Februar 2026 von <https://www.bast.de/DE/Forschung/Versuchseinrichtungen/U/Fahrsimulator.html>
- Bundesanstalt für Straßen- und Verkehrswesen. (kein Datum). *Mikroskopische Verkehrsflusssimulation auf einbahnigen Landstraßen*. Abgerufen am 29. Dezember 2025 von <https://www.bast.de/DE/Publikationen/Foko/2024-2023/2024-09.html>
- Daniel Krajzewicz, J. E. (kein Datum). *Recent Development and Applications of SUMO – Simulation of Urban MObility*. Abgerufen am 27. Dezember 2025 von [https://eclipse.dev/sumo/documents/sysmea\\_v5\\_n34\\_2012\\_4.pdf](https://eclipse.dev/sumo/documents/sysmea_v5_n34_2012_4.pdf)
- Docker. (kein Datum). *Install Docker Desktop on Windows*. Abgerufen am 6. Januar 2026 von <https://docs.docker.com/desktop/setup/install/windows-install/>
- Finstad, K. (06. Mai 2010). *The Usability Metric for User Experience*. Von <https://academic.oup.com/iwc/article-abstract/22/5/323/682940?redirectedFrom=fulltext> abgerufen
- Fraunhofer ITWM. (kein Datum). *Verkehrssimulation*. Abgerufen am 27. Dezember 2025 von <https://www.itwm.fraunhofer.de/de/abteilungen/mf/dynamik-systemsimulation/verkehr-simulation.html>

Hevner, A. R. (2004). *Design Science in Information Systems Research*. Abgerufen am 12. April 2026 von

[https://wise.vub.ac.be/sites/default/files/thesis\\_info/design\\_science.pdf](https://wise.vub.ac.be/sites/default/files/thesis_info/design_science.pdf)

Hochschule Liechtenstein. (kein Datum). *Simulation informationsorientierter Verkehrssysteme*. Abgerufen am 27. Dezember 2025 von

<https://ethz.ch/content/dam/ethz/special-interest/baug/ivt/ivt-dam/vpl/reports/601-700/ab606.pdf>

Kim, Y. (kein Datum). *Hardware-In-the-Loop for Connected Automated Vehicles Testing inReal Traffic*. Abgerufen am 4. Januar 2026 von <https://arxiv.org/pdf/1907.09052>

maptiler. (kein Datum). *Maps for developers*. Abgerufen am 26. Februar 2026 von <https://www.maptiler.com/>

Mo, T. (kein Datum). *Web-based Visualization and Interaction of SUMO Traffic Simulation in Digital Twin Model*. Abgerufen am 14. März 2026 von <https://kth.diva-portal.org/smash/get/diva2:1906701/FULLTEXT01.pdf>

PTV Group. (kein Datum). *Verkehrssimulation Software*. Abgerufen am 20. Dezember 2025 von <https://www.ptvgroup.com/de/produkte/ptv-vissim>

React. (kein Datum). *Eine JavaScript Bibliothek zum Erstellen von Benutzeroberflächen*. Abgerufen am 20. November 2025 von <https://de.legacy.reactjs.org/>

Reinold, D.-I. P. (kein Datum). *Verkehrssimulation im Hardware-in-the-Loop-Steuergerätetest*. Abgerufen am 2. Januar 2026 von <https://www.tkn.tu-berlin.de/bib/reinold2018verkehrssimulation/reinold2018verkehrssimulation.pdf>

Simulation of Urban MObility. (kein Datum). *SUMO*. Abgerufen am 22. Dezember 2025 von [https://eclipse.dev/sumo/documents/simul\\_2011\\_3\\_40\\_50150.pdf](https://eclipse.dev/sumo/documents/simul_2011_3_40_50150.pdf)

Simulation of Urban MObility. (kein Datum). *SUMO User Documentation*. Abgerufen am 29. Dezember 2025 von <https://sumo.dlr.de/docs/index.html>

Svenja Hilbrich. (kein Datum). *Entwicklung webbasierter graphischer  
Benutzeroberflächen für Simulationskerne*. Abgerufen am 24. Februar 2026 von  
<https://www.cea-wismar.de/asimsst/data/005-ASIM-SST-2020-hilbrich.pdf>

Tellmann, D. (kein Datum). *Hardware-in-the-Loop-gestützte Entwicklungsplattform für  
Fahrerassistenzsysteme*. Abgerufen am 20. Januar 2026 von [https://www.uni-  
kassel.de/upress/online/frei/978-3-86219-266-3.volltext.frei.pdf](https://www.uni-kassel.de/upress/online/frei/978-3-86219-266-3.volltext.frei.pdf)

## Eidesstattliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst habe, dass ich sie zuvor an keiner anderen Hochschule und in keinem anderen Studiengang als Prüfungsleistung eingereicht habe und dass ich keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder aus anderweitigen fremden Äußerungen entnommen wurden, sind als solche kenntlich gemacht.

26. Mai 2026

---

Ort, Datum

Nam Nguyen

---

Unterschrift